ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH

PHÂN TÍCH & THIẾT KẾ THUẬT TOÁN

Bài tập lớn: Minh họa thuật toán quay lui qua bài toán Sudoku

---

Sinh viên thực hiện : Lã Xuân Hồng
MSSV : 25410056
Lớp : CS112.F31.LT.TTNT
Giảng viên hướng dẫn : Huỳnh Thị Thanh Thương
Ngày nộp bài : 18 tháng 10 năm 2026

TP. Hồ Chí Minh, ngày 18 tháng 10 năm 2026

Mục lục

1 Giới thiệu bài toán **3**

1.1 Lý do chọn bài toán . . . 3
1.2 Nguồn gốc của Sudoku . . . 3
1.3 Ứng dụng thực tế . . . 4

2 Mô tả bài toán **5**

3 Mô hình bài toán **7**

3.1 Input . . . 7
3.2 Output . . . 7
3.3 Điều kiện và ràng buộc . . . 7
3.4 Ví dụ Input và Output . . . 8
3.5 Hai phần mở rộng của website . . . 9

4 Thiết kế cấu trúc dữ liệu và thuật toán **10**

4.1 Phân tích đặc trưng và ý tưởng giải . . . 10
4.2 Cấu trúc dữ liệu . . . 11
4.3 Thuật toán quay lui theo mô hình 1 . . . 13
4.4 Phân tích độ phức tạp . . . 14
4.5 Các biến thể đã phát triển thêm . . . 16
4.6 Lưới kích thước khác: 4×4 và 16×16 . . . 17
4.7 Cấu trúc dữ liệu của phần minh họa từng bước . . . 19

5 Ví dụ minh họa các bước thực hiện **21**

5.1 Chọn đề và cách đọc bảng . . . 21
5.2 Diễn biến . . . 23
5.3 Ví dụ thứ hai: quay lui qua sáu ô . . . 27
5.4 Bảng toàn bộ các bước . . . 31

6 Quá trình thiết kế và xây dựng **33**

6.1 Công cụ và cách làm việc . . . 33
6.2 Chọn đề và lập kế hoạch (3/10) . . . 35

6.3 Thiết kế giao diện bằng Claude Design (3/10 - 4/10) . . . 36
6.4 Viết bộ giải bằng TypeScript (4/10) . . . 37
6.5 Xây dựng trang Giải Sudoku (4/10) . . . 38
6.6 Một phiên Claude riêng: viết lại chữ và đóng vai người dùng (4/10) . . . 40
6.7 Yêu cầu chính thức và đổi góc nhìn (6/10) . . . 42
6.8 Bộ testcase (6/10) . . . 42
6.9 Trang So sánh (6/10) . . . 43
6.10 Trang Mọi lời giải (6/10) . . . 44
6.11 Quy tắc văn phong và viết lại chữ trên website (6/10) . . . 45
6.12 Đề minh họa và sự cố máy chậm (8/10 - 10/10) . . . 46
6.13 Viết report và Hội đồng AI (12/10) . . . 46
6.14 Đưa website lên Vercel và GitHub (12/10) . . . 49
6.15 Video demo bằng Playwright (12/10) . . . 50
6.16 Ba phần bổ sung sau Hội đồng AI (13/10) . . . 53
6.17 Hội đồng AI lần hai trước khi nộp (13/10) . . . 54
6.18 Phần việc của em và phần việc của AI . . . 54

7 Thiết kế thực nghiệm **57**
7.1 Cách tạo bộ testcase . . . 57
7.2 Các nhóm testcase . . . 58
7.3 Cách bảo đảm độ phủ . . . 59
7.4 Đáp án lấy từ đâu . . . 60

8 Đánh giá **61**
8.1 Kết quả trên bộ testcase . . . 61
8.2 Thời gian thực hiện . . . 61
8.3 So sánh các cách giải . . . 62
8.4 Kiểm thử tự động . . . 63
8.5 Tính trực quan của giao diện . . . 63
8.6 Hạn chế và hướng phát triển . . . 64

A Chạy mã nguồn **66**
A.1 Cấu trúc bài nộp . . . 66
A.2 Địa chỉ website và mã nguồn . . . 66
A.3 Các lệnh chạy . . . 67

Chương 1

Giới thiệu bài toán

1.1 Lý do chọn bài toán

Em phân vân giữa hai bài: tìm đường ngắn nhất bằng Dijkstra và giải Sudoku. Em thích cả hai, nên hỏi ý kiến Claude Code. Claude khuyên không chọn Dijkstra vì trên lớp chưa học bài này, cô chỉ nhắc tới khi cho xem sản phẩm của một sinh viên lớp chính quy. Một sản phẩm gần giống đã có sẵn thì bài của em dễ bị so sánh trực tiếp. Claude khuyên chọn Sudoku vì hai lý do, và em chọn Sudoku theo lời khuyên đó.

Lý do đầu là chiến lược giải. Sudoku giải bằng quay lui, phần cô dặn đề thi lúc nào cũng có. Làm bài này, em phải hiểu kỹ mô hình 1 trong slide của cô [1].

Lý do còn lại: thuật toán hiện ra được trên hình. Ba động tác thử số, loại số trùng và lùi về ô trước đều thấy ngay trên lưới. Yêu cầu của bài tập lớn là cho người dùng theo dõi từng bước, và lưới Sudoku làm được điều đó rõ hơn một đồ thị. Mã giả mô hình 1 cũng đặt được ngay cạnh lưới, tô sáng dòng đang chạy.

Các phần so sánh nhiều cách giải và tìm mọi lời giải đến sau, khi cô ra yêu cầu mới ngày 6/10 (Chương 6).

1.2 Nguồn gốc của Sudoku

Trò chơi lưới số dạng hiện nay do Howard Garns soạn, xuất hiện lần đầu năm 1979 trên tạp chí đố vui của Dell ở Mỹ với tên "Number Place". Năm 1984, nhà xuất bản Nikoli đưa trò chơi sang Nhật và đặt tên Sudoku, rút gọn từ một câu tiếng Nhật nghĩa là "mỗi chữ số chỉ được xuất hiện một lần". Từ năm 2004, khi báo The Times ở Anh đăng Sudoku hằng ngày, trò chơi lan ra khắp thế giới [5].

Luật chơi chỉ có một câu: điền lưới 9×9 sao cho mỗi hàng, mỗi cột và mỗi khối 3×3 có đủ các số từ 1 đến 9. Vậy mà số lưới đầy hợp lệ lên tới 6.670.903.752.021.072.936.960, theo phép tính của Felgenhauer và Jarvis năm 2005. Một đề có lời giải duy nhất cần ít nhất 17 số cho sẵn, McGuire và cộng sự chứng minh năm 2012 [6]. Bộ testcase của em có một đề 17 số lấy từ danh

sách của Gordon Royle.

1.3 Ứng dụng thực tế

Sudoku là một bài toán thỏa mãn ràng buộc: có 81 biến (các ô), mỗi biến nhận giá trị trong miền $\{1, \ldots, 9\}$, và các ràng buộc "khác nhau" trong từng hàng, cột, khối. Cùng khung này xuất hiện trong nhiều bài toán thật, vì các bài sau đều đưa được về tô màu đồ thị [7]:

- **Xếp lịch thi.** Hai môn có chung sinh viên không được xếp cùng ca, giống hai ô cùng hàng không được cùng số.
- **Xếp thời khóa biểu.** Một giáo viên không dạy hai lớp cùng tiết, một phòng không chứa hai lớp cùng tiết.
- **Tô màu đồ thị.** Mỗi ô là một đỉnh. Hai ô cùng hàng, cột hoặc khối nối với nhau bằng một cạnh. Mỗi đỉnh có đúng 20 đỉnh kề: 8 ô cùng hàng, 8 ô cùng cột, 4 ô còn lại cùng khối. Giải Sudoku chính là tô đồ thị 81 đỉnh, 810 cạnh bằng 9 màu sao cho hai đỉnh kề khác màu, với một số đỉnh đã tô sẵn.

Quay lui cũng là cách giải chung cho các bài cùng loại trong slide của cô: 8 quân hậu, tổng tập con, chu trình Hamilton, tô màu đồ thị [1]. Với Sudoku tổng quát cỡ $n^2 \times n^2$, bài toán thuộc lớp NP-đầy đủ [6], nên chưa ai biết cách giải luôn nhanh cho mọi đề. Với lưới 9×9 cố định, đề "chống quay lui" trong bộ testcase của em là một ví dụ về đề làm thuật toán chậm: chỉ 64 ô trống mà cần 622.577.597 lần thử.

Bản thân website là một ứng dụng dạy học. Người mới học thuật toán nhìn được thuật toán thử, loại và lùi lại trên từng ô, thay vì chỉ đọc mã giả.

Chương 2

Mô tả bài toán

Sudoku là trò chơi điền số trên lưới 9×9. Lưới chia thành 9 khối 3×3 bằng các đường kẻ đậm. Một lưới Sudoku hợp lệ khi mỗi hàng, mỗi cột và mỗi khối đều chứa đủ 9 số từ 1 đến 9, mỗi số đúng một lần.

Cho một lưới trong đó một số ô đã điền sẵn, các ô còn lại để trống. Hãy điền số vào các ô trống để được một lưới Sudoku hợp lệ, giữ nguyên các số đã cho.

Input

Gồm 9 dòng, mỗi dòng 9 ký tự cách nhau bởi dấu cách. Mỗi ký tự là một số từ 1 đến 9 (ô đã điền) hoặc chữ `x` (ô trống). Số ô trống từ 0 đến 81.

Output

- Nếu điền được: 9 dòng, mỗi dòng 9 số cách nhau bởi dấu cách, là lưới đã điền kín và hợp lệ. Nếu có nhiều cách điền, xuất cách đầu tiên tìm được khi xét ô trống theo thứ tự hàng rồi cột và thử số từ nhỏ đến lớn.
- Nếu đề có hai ô cùng số trong cùng một hàng, cột hoặc khối: thông báo đề không hợp lệ, kèm số trùng và vị trí.
- Nếu không có cách điền nào: thông báo đề không có lời giải.

Ví dụ

Đề dưới đây là đề số 13, tên "Ngắn, có quay lui", trong danh sách đề mẫu của website.

Input

```
5 3 4 6 7 X 9 X X
6 7 X X 9 5 3 4 8
X 9 8 3 4 X 5 6 7
8 X 9 7 6 X X 2 3
4 X 6 X 5 3 7 9 1
7 1 3 9 X 4 8 5 6
9 6 1 5 3 7 2 X 4
2 8 7 4 1 9 6 3 5
X 4 5 X 8 6 1 7 9
```

Output

```
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```

Yêu cầu

Giải bằng kỹ thuật quay lui theo mô hình 1. Chương trình phải cho người dùng nhập đề, theo dõi từng bước thuật toán thực hiện, và xem kết quả.

Chương 3

Mô hình bài toán

3.1 Input

- A: ma trận 9×9 ký tự. $A[r][c] \in \{1, \ldots, 9, \texttt{X}\}$ với $1 \le r, c \le 9$. $A[r][c]$ là số đề cho ở ô hàng r, cột c; X nghĩa là ô đó trống.

Ngoài cách viết trên, hộp Dán đề của website nhận thêm hai cách: 9 dòng viết liền không dấu cách, và một chuỗi 81 ký tự trong đó dấu chấm là ô trống. Cả ba cách cho cùng ma trận A.

3.2 Output

Một trong ba kết quả:

- B: ma trận 9×9 số nguyên, $B[r][c] \in \{1, \ldots, 9\}$, là lưới đã điền kín và hợp lệ.
- M_1: chuỗi ký tự báo đề không hợp lệ, gồm số bị trùng, hàng, cột hay khối chứa số đó, và vị trí các ô trùng.
- M_2: chuỗi ký tự báo đề không có lời giải. Nếu ô trống đầu tiên đã hết số ngay từ đầu, chuỗi nêu ô đó và lý do từng số bị loại; nếu không, chuỗi nêu số lần thử đã dùng.

3.3 Điều kiện và ràng buộc

- Số ô trống n thỏa $0 \le n \le 81$.
- Khối thứ b, với $1 \le b \le 9$, gồm các ô có hàng từ $3\lfloor (b-1)/3 \rfloor + 1$ đến $3\lfloor (b-1)/3 \rfloor + 3$ và cột từ $3\left((b-1) \bmod 3\right) + 1$ đến $3\left((b-1) \bmod 3\right) + 3$.
- **Đề hợp lệ** khi không có hai ô đã điền nào cùng số mà nằm chung hàng, chung cột hoặc chung khối.

- **Giữ nguyên số đề cho:** $B[r][c] = A[r][c]$ với mọi ô có $A[r][c] \neq \text{x}$.
- **Lưới hợp lệ:** mỗi hàng r có $\{B[r][1], \ldots, B[r][9]\} = \{1, \ldots, 9\}$. Mỗi cột và mỗi khối cũng vậy.
- **Nhiều lời giải:** chọn lời giải đầu tiên tìm được khi xét ô trống theo thứ tự hàng rồi cột, thử số từ 1 đến 9. Nhờ quy ước này, mỗi đề chỉ có một đầu ra đúng, nên đầu ra của website so được với bản cài đặt tham chiếu đến từng ký tự.

3.4 Ví dụ Input và Output

Ví dụ 1: đề có lời giải duy nhất

Đề ở mục Ví dụ của Chương 2, tức đề số 13 "Ngắn, có quay lui" trong danh sách đề mẫu của website. Chương 5 đi qua toàn bộ các bước giải đề này.

Ví dụ 2: đề nhiều lời giải

Đề "Nhiều lời giải" có 44 ô trống và 18 lời giải. Output là lời giải đầu tiên, thuật toán tìm ra sau 827 lần thử.

Input

```
5 3 X 6 X X 9 1 2
X X X X 9 5 3 4 X
1 9 X X X 2 X 6 X
X X X 7 6 1 X 2 X
X X 6 X X 3 7 X X
X 1 X 9 2 X 8 X X
X X 1 X X X X 8 4
X X X 4 1 X X 3 5
X 4 X 2 X 6 1 7 X
```

Output

```
5 3 4 6 7 8 9 1 2
2 6 7 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
6 7 1 5 3 9 2 8 4
9 8 2 4 1 7 6 3 5
3 4 5 2 8 6 1 7 9
```

Ví dụ 3: đề không hợp lệ

Hai số 7 cùng nằm trên hàng 4.

Input

```
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
7 X X X X X X X 7
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
```

Output

Đề không hợp lệ: số 7 xuất hiện 2 lần ở hàng 4 (cột 1 và cột 9).

Ví dụ 4: đề không có lời giải

Hàng 1 đã có các số từ 2 đến 9, cột 1 đã có số 1, nên ô $(1, 1)$ không nhận được số nào.

Input

```
X 2 3 4 5 6 7 8 9
1 X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
X X X X X X X X X
```

Output

Đề này không có lời giải. Ở ô $(1, 1)$, cả 9 số đều trùng: số 1 đã có ở cột 1, các số từ 2 đến 9 đã có ở hàng 1.

3.5 Hai phần mở rộng của website

Website còn có hai phần mở rộng, đều dùng lại mô hình bài toán ở trên với một thay đổi (Chương 4).

- **Tìm mọi lời giải.** Input giữ nguyên. Output thay B bằng danh sách $B_1, B_2, \ldots, B_m$, là mọi lời giải theo thứ tự tìm được, và số lời giải $m \geq 0$. Đề không có lời giải thì $m = 0$.
- **Lưới $k \times k$.** Với $k = b^2$, A là ma trận $k \times k$. Mỗi phần tử là ô trống hoặc một trong k chữ số đầu của dãy $1, \ldots, 9, \texttt{A}, \ldots, \texttt{G}$, trong đó `A` là 10 và `G` là 16. Khối gồm $b \times b$ ô và số ô trống $n \leq k^2$. Lưới hợp lệ khi mỗi hàng, mỗi cột và mỗi khối có đủ k giá trị, mỗi giá trị một lần. Các cỡ dùng trên website là $k = 4, 9, 16$, ứng với $b = 2, 3, 4$.

Chương 4

Thiết kế cấu trúc dữ liệu và thuật toán

Chiến lược giải là **quay lui** (Backtracking) theo **mô hình 1** trong slide của cô [1], cùng ý với mục quay lui trong giáo trình của Levitin [2]. Chương này đi theo thứ tự cô hướng dẫn cho câu thiết kế: phân tích đặc trưng để chọn chiến lược, mô hình hóa cấu trúc lời giải, chọn cấu trúc dữ liệu, viết mã giả, rồi tính độ phức tạp.

4.1 Phân tích đặc trưng và ý tưởng giải

Bài toán có đủ hai dấu hiệu của quay lui:

- **Lời giải gồm nhiều bước.** Mỗi ô trống là một bước. Đề Arto Inkala, một đề nổi tiếng khó (Chương 7), có 60 ô trống, tức 60 bước.
- **Mỗi bước có nhiều lựa chọn.** Mỗi ô trống có 9 số để chọn, từ 1 đến 9.

Không có tiêu chí "tốt nhất" nào cần tối ưu, chỉ cần một cách điền đúng luật. Vậy đây là bài toán quyết định, hợp với quay lui hơn nhánh cận.

Mô hình hóa cấu trúc lời giải

- Đánh số các ô trống lần lượt từ trái sang phải, từ trên xuống dưới: ô trống thứ $1, 2, \ldots, n$.
- Lời giải là vectơ $S = (S_1, S_2, \ldots, S_n)$. $S_i \in \{1, \ldots, 9\}$ là số điền vào ô trống thứ i.
- n là số thành phần của lời giải, bằng số ô trống. $k = 9$ là số khả năng mà mỗi S_i có thể chọn.
- Thứ tự các thành phần có nghĩa: S_i gắn với đúng một ô cố định trên lưới.
- **Khả năng $S_i = j$ chấp nhận được** khi số j chưa có ở hàng, ở cột và ở khối 3×3 chứa ô trống thứ i.

Ý tưởng

Thuật toán điền lần lượt từng ô trống. Ở ô trống thứ i, thuật toán thử $j = 1, 2, \ldots, 9$. Mỗi lần kiểm tra một số j ở một ô, dù số đó được chấp nhận hay bị loại, là một **lần thử**. Số nào trùng hàng, cột hoặc khối thì bỏ ngay, nên cả nhánh phía sau số đó không bao giờ phải thử. Số nào chấp nhận được thì điền vào rồi sang ô trống thứ $i + 1$. Nếu ở một ô cả 9 số đều không đi tiếp được, thuật toán lùi về ô trước, xóa số đang điền ở đó và thử số kế tiếp.

Các lần thử tạo thành một cây tìm kiếm. Mỗi lời gọi `try(i)` là một nút, mỗi số j chấp nhận được mở ra một nhánh con. Lùi về ô trước nghĩa là quay lên nút cha.

Đề số 13 trong Chương 5 cho thấy ý tưởng này trên lưới thật. Thuật toán điền số 1 vào ô $(1, 6)$. Ở ô kế tiếp $(1, 8)$, cả 9 số đều trùng, nên thuật toán lùi về ô $(1, 6)$, xóa số 1 và thử số 2.

4.2 Cấu trúc dữ liệu

Bảng 4.1 liệt kê các biến của thuật toán, kiểu, miền giá trị và vai trò.

Bảng 4.1: Cấu trúc dữ liệu của thuật toán quay lui

Biến	**Kiểu, miền giá trị**	**Vai trò**
`grid`	Ma trận 9×9 số nguyên trong $[0, 9]$	Trạng thái lưới hiện tại. Số 0 là ô trống
`emptyCells`	Mảng n cặp (hàng, cột)	Ô trống thứ i nằm ở đâu, theo thứ tự hàng rồi cột
n	Số nguyên, $0 \le n \le 81$	Số ô trống, cũng là số thành phần của lời giải
$S[1..n]$	Mảng n số nguyên trong $[1, 9]$	Vectơ lời giải $S = (S_1, \ldots, S_n)$
`rowUsed`	Ma trận 9×9 giá trị đúng/sai	`rowUsed[r][d]` đúng khi hàng r đã có số d
`columnUsed`	Ma trận 9×9 giá trị đúng/sai	`columnUsed[c][d]` đúng khi cột c đã có số d
`boxUsed`	Ma trận 9×9 giá trị đúng/sai	`boxUsed[b][d]` đúng khi khối b đã có số d
i	Số nguyên trong $[1, n]$	Bước đang xét, tức ô trống thứ i
j	Số nguyên trong $[1, 9]$	Số đang thử cho ô trống thứ i
(r, c)	Cặp số nguyên trong $[1, 9]$	Hàng và cột của ô trống thứ i, lấy từ `emptyCells[i]`
$b(r, c)$	Hàm, giá trị trong $[1, 9]$	Số thứ tự của khối chứa ô (r, c)
`found`	Giá trị đúng/sai	Đúng khi đã xuất lời giải, để mọi tầng đệ quy dừng lại

Khối chứa ô (r, c), với hàng và cột đánh số từ 1, là

$$b(r, c) = 3 \left\lfloor \frac{r-1}{3} \right\rfloor + \left\lfloor \frac{c-1}{3} \right\rfloor + 1.$$

Các khối đánh số từ 1 đến 9, từ trái sang phải rồi từ trên xuống. Ví dụ ô $(1, 6)$ nằm ở khối $b = 0 + 1 + 1 = 2$, ô $(1, 8)$ ở khối 3, góc trên bên phải.

Ba mảng đánh dấu làm mỗi lần kiểm tra rẻ đi. Không có chúng, mỗi lần kiểm tra số j phải đọc lại cả hàng, cả cột và cả khối, tới 27 ô. Có chúng, chỉ cần đọc ba giá trị `rowUsed[r][j]`, `columnUsed[c][j]`, `boxUsed[b][j]`, và mỗi lần điền hay xóa một số chỉ cần sửa ba giá trị. Trên đề Inkala, bộ giải nhờ vậy chạy nhanh gấp khoảng hai lần (Chương 8). Đổi lại, ba mảng tốn $3 \times 9 \times 9 = 243$ giá trị đúng/sai, một lượng bộ nhớ cố định không phụ thuộc vào đề.

4.3 Thuật toán quay lui theo mô hình 1

Ở mỗi bước i, mô hình 1 thử lần lượt mọi khả năng j. Khả năng nào chấp nhận được thì mô hình ghi nhận rồi đi tiếp sang bước $i+1$, quay về thì trả lại trạng thái cũ. Mã giả trong slide của cô:

```
try (i) =
{     for (j = 1 --> k)   // k: số khả năng mà S_i có thể chọn
           if (khả năng j chấp nhận được)
           {   Xác định S_i theo j;
               Ghi nhận trạng thái mới của bài toán (nếu có);
               if (i < n)
                     try (i+1); // tiến hành bước i+1 để xác định tiếp S_{i+1}
               else
                     Xuất lời giải S = (S_1, S_2, …, S_n);
               Trả lại trạng thái cũ của bài toán;
           }
}
```

Áp dụng cho Sudoku, thuật toán có ba bước.

Bước 1. Chuẩn bị. Đọc lưới. Với mỗi ô (r, c) có sẵn số d, bật cờ của số d ở hàng r, cột c và khối $b(r, c)$ trong ba mảng đánh dấu. Với mỗi ô trống, thêm (r, c) vào cuối `emptyCells`. Đặt n bằng số ô trống và `found = false`.

Bước 2. Kiểm tra đề. Nếu một số xuất hiện hai lần trong cùng hàng, cột hoặc khối thì báo đề không hợp lệ và dừng. Nếu $n = 0$ thì lưới đã đầy, không cần giải.

Bước 3. Gọi `try(1)`. Nếu `try(1)` chạy xong mà `found` vẫn sai thì đề vô nghiệm.

```
try (i) =
{     (r, c) = emptyCells[i];
      for (j = 1 --> 9)
           if (!rowUsed[r][j] && !columnUsed[c][j] && !boxUsed[b(r,c)][j])
           {   S[i] = j;
               grid[r][c] = j;
               rowUsed[r][j] = columnUsed[c][j] = boxUsed[b(r,c)][j] = true;
               if (i < n)
                     try (i+1);
               else
               {     Xuất lời giải grid;
                     found = true;
               }
               if (found) return;
               grid[r][c] = 0;
               rowUsed[r][j] = columnUsed[c][j] = boxUsed[b(r,c)][j] = false;
```

```
            }
}
```

Bản dành cho Sudoku do Claude Code viết theo mô hình 1. Bảng 4.2 đối chiếu từng dòng mô hình 1 với bản này.

Bảng 4.2: Đối chiếu mô hình 1 với bản dành cho Sudoku

Mô hình 1	**Với Sudoku**
`for (j = 1 --> k)`	Thử j từ 1 đến 9, vì $k = 9$
`if (khả năng j chấp nhận được)`	Hàng, cột và khối của ô trống thứ i đều chưa có số j, kiểm tra qua ba mảng đánh dấu
`Xác định` S_i `theo j`	`S[i] = j`
`Ghi nhận trạng thái mới`	Ghi j vào lưới, bật ba cờ đánh dấu
`if (i < n) try(i+1)`	Sang ô trống kế tiếp
`Xuất lời giải`	Xuất lưới đã điền kín, đặt `found = true`
Không có	`if (found) return;`
`Trả lại trạng thái cũ`	Xóa j khỏi lưới, tắt ba cờ đánh dấu

Bản dành cho Sudoku khác mô hình 1 ở một chỗ. Bản này có thêm biến `found` và dòng `if (found) return;`. Khi xuất lời giải đầu tiên, `found` thành đúng. Mỗi tầng `try` vừa quay về đều gặp dòng này và thoát ngay, trước dòng trả lại trạng thái cũ, nên lưới vẫn giữ nguyên lời giải. Đề bài chỉ cần một cách điền, nên đi tiếp là thừa. Trên website, hàm `try` trả về đúng hoặc sai thay cho biến `found`, cùng một ý. Trang "Mọi lời giải" trên website chạy đúng nguyên văn mô hình 1, không dừng, để đếm đề có bao nhiêu lời giải (mục 4.5).

4.4 Phân tích độ phức tạp

Gọi $T(n)$ là chi phí khi còn n ô trống. Mỗi lần gọi `try` thử 9 số. Mỗi lần kiểm tra chỉ đọc ba mảng đánh dấu, mỗi lần ghi nhận hay trả lại chỉ sửa ba giá trị, nên phần việc ngoài các lời gọi đệ quy tốn một hằng số c_2. Mỗi số chấp nhận được mở ra một bài toán con còn $n - 1$ ô trống. Để có cận trên, giả sử ở mọi tầng cả 9 số đều chấp nhận được. Thực tế không như vậy: số vừa điền ở một ô đã chặn số đó ở mọi ô cùng hàng, cùng cột, cùng khối. Nhưng chi phí thật không thể vượt quá chi phí trong giả sử đó:

$$T(n) = 9\,T(n-1) + c_2, \qquad T(0) = c_1.$$

c_1 là chi phí khi đã điền hết ô trống: xuất lời giải.

Giải bằng phương pháp truy hồi:

$$\begin{aligned} T(n) &= 9\,T(n-1) + c_2 \\ &= 9\,[9\,T(n-2) + c_2] + c_2 = 9^2\,T(n-2) + 9c_2 + c_2 \\ &= 9^2\,[9\,T(n-3) + c_2] + 9c_2 + c_2 = 9^3\,T(n-3) + (9^2 + 9 + 1)\,c_2 \\ &= \dots \\ &= 9^i\,T(n-i) + c_2 \sum_{t=0}^{i-1} 9^t \qquad (*) \end{aligned}$$

Quá trình dừng lại khi $n - i = 0 \iff i = n$. Ta có công thức tổng cấp số nhân $\sum_{t=0}^{m} a^t = \dfrac{a^{m+1} - 1}{a - 1}$. Với $a = 9$, $m = i - 1$: $\sum_{t=0}^{i-1} 9^t = \dfrac{9^i - 1}{9 - 1} = \dfrac{9^i - 1}{8}$. Thay $i = n$ vào $(*)$:

$$T(n) = 9^n\,T(0) + c_2\,\frac{9^n - 1}{8} = \left(c_1 + \frac{c_2}{8}\right) 9^n - \frac{c_2}{8} \in \mathcal{O}(9^n)\,.$$

Con số 9^n lớn tới mức nào? Đề Arto Inkala có $n = 60$, nên cận trên là 9^{60}, một số có 58 chữ số. Thuật toán thực tế chỉ cần 445.778 lần thử cho đề này. Khoảng cách đó đến từ việc bỏ sớm: số nào trùng thì thuật toán cắt luôn cả nhánh phía sau.

Có đề làm số lần thử tăng vọt. Đề "chống quay lui" trong bộ testcase (Chương 7) để trống cả hàng 1, mà lời giải của hàng này là 9 8 7 6 5 4 3 2 1. Ở mỗi ô của hàng 1, số đúng là số lớn nhất còn lại, nên thuật toán thử số đó sau cùng. Kết quả: 622.577.597 lần thử với 64 ô trống, gấp khoảng 1.400 lần đề Inkala. Con số đó vẫn còn cách xa cận trên, vì 9^{64} là một số có 62 chữ số.

Về bộ nhớ, ba mảng đánh dấu và lưới có kích thước cố định. Vectơ S, danh sách ô trống và độ sâu đệ quy cùng tỉ lệ với n. Tổng bộ nhớ là $\mathcal{O}(n)$, với $n \le 81$.

Trường hợp tốt nhất

Cận $\mathcal{O}(9^n)$ là trường hợp xấu nhất. Trường hợp tốt nhất xảy ra khi đề có lời giải và thuật toán không quay lui lần nào: mỗi ô trống i loại các số nhỏ hơn S_i rồi nhận S_i, và nhánh phía sau luôn đi tiếp được. Số lần thử khi đó là

$$S_1 + S_2 + \cdots + S_n \le 9n.$$

Gọi c_3 là chi phí của một lần thử. Chi phí tổng là $T(n) = c_1 + c_3\,(S_1 + \cdots + S_n)$, mà $n \le S_1 + \cdots + S_n \le 9n$, nên $c_1 + c_3\,n \le T(n) \le c_1 + 9c_3\,n$, tức $T(n) \in \Theta(n)$: tăng tuyến tính theo số ô trống. Ví dụ ở Chương 5 nếu không lùi lần nào thì cần $S_1 + \cdots + S_{16} = 52$ lần thử, và đề "Dễ" (5 ô trống, 0 lần trả lại) cần 27 lần thử. Phần chuẩn bị đọc lưới, bật cờ cho các số đã cho và kiểm tra đề chỉ duyệt 81 ô, tốn một hằng số, không đổi theo n.

4.5 Các biến thể đã phát triển thêm

Ngoài bản chính, website có ba nhóm biến thể để so sánh: cách kiểm tra "chấp nhận được", thứ tự chọn ô, và việc dừng hay không dừng sau lời giải đầu tiên. Cả ba vẫn giữ khung mô hình 1. Mỗi biến thể có một tên trong report và một tên trên website, ghi ngay chỗ giới thiệu.

Ba cách kiểm tra "chấp nhận được"

Thứ tự xét ô giữ nguyên, chỉ đổi cấu trúc dữ liệu dùng để kiểm tra.

- **Quét lưới** (trên website: "Nhìn lại lưới"). Không dùng mảng phụ. Mỗi lần kiểm tra đọc lại hàng, cột và khối, tới 27 ô.
- **Ba mảng đánh dấu** (trên website: "Sổ ghi chép"). Cách của bản chính, đọc tối đa 3 giá trị.
- **Bitmask** (trên website: "Dãy công tắc"). Mỗi hàng, cột, khối gói vào một số nguyên 9 bit, bit thứ d tính từ bên phải bật khi số d đã có. Hàng đã có số 1 và 3 thì số của hàng, viết ở hệ nhị phân, là `000000101`. Để kiểm tra số j, gộp ba số của hàng, cột, khối bằng hai phép OR, rồi AND với số chỉ bật bit thứ j. Kết quả khác 0 nghĩa là trùng. Với hàng trên, `000000101` AND `000000100` khác 0, nên số 3 trùng.

Ba cách cho đúng cùng cây tìm kiếm, nên cùng số lần thử trên mọi đề. Cái khác là hằng số c_2 trong $T(n) = 9\,T(n-1) + c_2$. Trên đề Arto Inkala, quét lưới mất 9,55 ms, ba mảng đánh dấu 5,17 ms, bitmask 4,67 ms (Chương 8).

Chọn ô ít khả năng nhất trước (MRV)

Thay vì lấy ô trống kế tiếp theo hàng rồi cột, trước mỗi bước thuật toán đếm số giá trị còn chấp nhận được của mọi ô trống chưa điền, rồi chọn ô có ít nhất. Hai ô bằng nhau thì lấy ô đứng trước theo hàng rồi cột. Cách này thường gọi là MRV (minimum remaining values) [3, 4]. Trên website, cách này mang tên "Điền ô dễ nhất trước", và các bảng so sánh gọi tắt là MRV. S_i lúc này là số điền vào ô được chọn ở bước i, không còn gắn với một ô cố định.

- **Cây nhỏ đi.** Đề Arto Inkala giảm từ 445.778 xuống 90.665 lần thử. Đề chống quay lui giảm từ 622.577.597 xuống 407.156.
- **Mỗi bước đắt hơn.** Mỗi lần chọn ô, thuật toán kiểm tra tối đa 9 số cho mỗi ô trống còn lại, tức $\mathcal{O}(n)$ thay cho $\mathcal{O}(1)$. Trên đề Inkala, tổng thời gian vì thế lại dài hơn bản chính: 23,5 ms so với 5,17 ms.
- **Cận trên không đổi.** Trường hợp xấu nhất vẫn là $\mathcal{O}(9^n)$.

Tìm mọi lời giải

Biến thể này (trên website: trang "Mọi lời giải") chạy đúng nguyên văn mô hình 1: sau khi xuất một lời giải, thuật toán trả lại trạng thái cũ và thử tiếp, tới khi hết mọi khả năng. Thuật toán luôn đi hết cây tìm kiếm. Với đề có lời giải, số lần thử vì vậy thường lớn hơn hẳn bản dừng sớm; với đề vô nghiệm, hai bản thử bằng nhau vì bản dừng sớm cũng phải đi hết cây. Với đề Arto Inkala, tìm lời giải đầu tiên cần 445.778 lần thử, còn đi hết để biết chắc chỉ có một lời giải cần 18.619.020 lần. Biến thể này cho biết đề mẫu "Vừa" có 4 lời giải và đề "Nhiều lời giải" có 18.

Độ phức tạp của các biến thể

Cả bốn mục dưới đây đều giữ khung $T(n) = 9\,T(n-1) + c$, chỉ đổi hằng số c, nên cận trên vẫn là $\mathcal{O}(9^n)$:

- **Ba cách kiểm tra:** quét lưới đọc tối đa 27 ô mỗi lần kiểm tra, ba mảng đánh dấu đọc 3 giá trị, bitmask đọc 3 số rồi gộp. Cả ba đều chỉ tốn một hằng số, khác nhau ở độ lớn của c_2.
- **Chọn ô ít khả năng nhất trước:** mỗi bước phải quét tối đa n ô trống, mỗi ô thử 9 số. Chi phí thêm này tối đa $9n$ phép kiểm tra, mà $n \le 81$, nên không quá 729 phép kiểm tra, mỗi phép tốn không quá c_2. Hằng số mới $c_2' \le 730\,c_2$ thay cho c_2 trong phương trình, kết quả vẫn $\mathcal{O}(9^n)$. Nếu cho n tăng không giới hạn thì chi phí thêm là $\mathcal{O}(n)$ mỗi bước, và $T(n) = 9\,T(n-1) + an$ vẫn cho $\mathcal{O}(9^n)$ vì 9^n lớn hơn hẳn mọi đa thức của n.
- **Tìm mọi lời giải:** thuật toán không dừng ở lời giải đầu nên luôn đi hết cây tìm kiếm. Mỗi lời gọi `try` thử đủ 9 số, nên số lần thử bằng 9 lần số nút của cây. Cận trên vẫn $\mathcal{O}(9^n)$. Trường hợp tốt nhất vẫn $\Omega(n)$: khi mỗi ô chỉ còn một số chấp nhận được, cây là một đường thẳng n nút và cần $9n$ lần thử, như đề "Dễ" (5 ô trống, 45 lần thử). Đề Arto Inkala cần 445.778 lần thử để tìm lời giải đầu tiên và 18.619.020 lần, đúng 9 lần 2.068.780 nút, để đi hết. Mỗi lời giải tìm được tốn thêm một hằng số để xuất lưới.
- **Generator của phần minh họa** (mục 4.7): mỗi sự kiện phải đi qua tối đa n tầng `yield*`, nên chi phí mỗi bước tăng từ c_2 lên $c_2 + c\,n$, cũng bị chặn bởi $n \le 81$. Bậc không đổi, nhưng hằng số đo được lớn hơn từ 63 đến 135 lần tùy đề (Chương 8).

4.6 Lưới kích thước khác: 4×4 và 16×16

Mô hình 1 không phụ thuộc kích thước lưới, chỉ phụ thuộc k, số khả năng của mỗi ô. Lưới có khối $b \times b$ thì cạnh lưới là $k = b^2$ và có tối đa k^2 ô trống. Lưới 4×4 có $b = 2$, lưới 9×9 có $b = 3$, lưới 16×16 có $b = 4$. Trên lưới 16×16, các số là 1 đến 9 rồi A đến G, với A là 10, B là 11, và cứ thế đến G là 16.

Phương trình truy hồi giữ nguyên dạng, chỉ thay 9 bằng k: $T(n) = k\,T(n-1)+c_2, T(0) = c_1$. Giải bằng phương pháp truy hồi:

$$\begin{aligned} T(n) &= k\,T(n-1) + c_2 \\ &= k\left[k\,T(n-2) + c_2\right] + c_2 = k^2\,T(n-2) + kc_2 + c_2 \\ &= k^2\left[k\,T(n-3) + c_2\right] + kc_2 + c_2 = k^3\,T(n-3) + (k^2 + k + 1)\,c_2 \\ &= \dots \\ &= k^i\,T(n-i) + c_2 \sum_{t=0}^{i-1} k^t \qquad (*) \end{aligned}$$

Quá trình dừng lại khi $n - i = 0 \iff i = n$. Với $k > 1$, ta có $\sum_{t=0}^{i-1} k^t = \dfrac{k^i - 1}{k-1}$. Thay $i = n$ vào $(*)$:

$$T(n) = k^n\,T(0) + c_2\,\frac{k^n - 1}{k-1} = \left(c_1 + \frac{c_2}{k-1}\right) k^n - \frac{c_2}{k-1} \in \mathcal{O}(k^n)\,.$$

Phần cấu trúc dữ liệu đổi ở hai chỗ. Ba mảng đánh dấu có kích thước $k \times k$ thay cho 9×9, nên lưới 16×16 tốn $3 \times 16 \times 16 = 768$ giá trị đúng/sai. Số thứ tự khối của ô (r, c) là $b\left\lfloor \frac{r-1}{b} \right\rfloor + \left\lfloor \frac{c-1}{b} \right\rfloor + 1$. Cách kiểm tra không đổi: hàng, cột và khối đều có k ô, nên mỗi lần kiểm tra vẫn đọc 3 giá trị. Website có trang riêng cho phần này, tên là Kích thước lưới, chạy đúng mã giả mô hình 1. Trang dùng một bộ giải tổng quát riêng, viết theo kích thước lưới thay cho hằng số 9 (Chương 6).

Bảng 4.3 ghi số lần thử của 11 đề mẫu, đo bằng bộ giải của website và so với bản Python riêng `checks/reference_sized_solver.py`. Cột k^n là số chữ số của cận trên.

Bảng 4.3: Số lần thử theo kích thước lưới (điền ô theo hàng rồi cột)

Cỡ	Đề	n	Chữ số của k^n	Lần thử	Lần trả lại	Lời giải
4×4	Dễ	6	4	11	0	1
4×4	Có quay lui	10	7	52	7	1
4×4	Nhiều lời giải	12	8	28	0	10
4×4	Vô nghiệm	10	7	44	10	0
4×4	Lưới trống	16	10	40	0	288
9×9	Ngắn, có quay lui	16	16	79	3	1
9×9	Vừa	40	39	1.909	187	4
16×16	Dễ	40	49	352	1	1
16×16	Vừa	70	85	1.111	30	1
16×16	Khó	100	121	26.006	1.573	9
16×16	Rất khó	120	145	406.824	25.364	36

Ba điều rút ra từ bảng:

- **Cận k^n lớn lên rất nhanh theo cả k và n.** Đề 16×16 "Rất khó" có $n = 120$ nên k^n có 145 chữ số, còn lưới 4×4 trống có $n = 16$ nên 4^{16} chỉ có 10 chữ số.
- **Số lần thử thật nhỏ hơn cận nhiều bậc, như ở lưới 9×9.** Lưới 4×4 trống cần 40 lần thử để tìm lời giải đầu tiên. Đếm đủ 288 lời giải của nó cần 9.860 lần thử. Đề 16×16 "Rất khó" cần 406.824 lần thử.
- **Số ô trống n không quyết định số lần thử.** Đề 16×16 "Dễ" có $n = 40$ và chỉ cần 352 lần thử, còn đề 9×9 "Vừa" cũng có $n = 40$ mà cần 1.909 lần. Đề 16×16 này cho sẵn 216 trên 256 ô, còn đề 9×9 cho sẵn 41 trên 81 ô.

Để chắc bộ giải tổng quát đúng, có hai kiểm tra. Trên 8 đề 9×9 mẫu, bộ giải tổng quát phát ra đúng cùng chuỗi sự kiện với bộ giải 9×9 của trang chính, từng sự kiện một. Trên 11 đề của Bảng 4.3, số lần thử, ghi nhận, trả lại và lời giải đầu tiên đều khớp bản Python. Các đề 16×16 do script `checks/find_sized_puzzles.py` sinh từ một lưới đầy trộn ngẫu nhiên rồi xóa ô, nên đề không nhất thiết có lời giải duy nhất. Đề "Khó" có 9 lời giải, đề "Rất khó" có 36. Thuật toán chỉ cần lời giải đầu tiên nên điều này không ảnh hưởng.

4.7 Cấu trúc dữ liệu của phần minh họa từng bước

Phần minh họa cần dừng thuật toán ở bất kỳ bước nào rồi chạy tiếp. Hàm đệ quy bình thường chạy một mạch nên không làm được việc đó. Trên website, `try` viết thành một *generator*: hàm có thể tạm dừng ở giữa và tiếp tục từ đúng chỗ đó. Ở các dòng chính của mô hình 1, generator phát ra một sự kiện, tức trả ra một bản ghi nhỏ cho giao diện rồi tạm dừng (Bảng 4.4).

Bảng 4.4: Sự kiện generator phát ra và cách giao diện hiển thị

Sự kiện	**Dòng mô hình 1**	**Hiển thị**
`acceptCandidate`	Khả năng j chấp nhận được	Ô nền vàng, mã giả gắn nhãn "đúng"
`rejectCandidate`	Khả năng j không chấp nhận được	Số gạch ngang, ô gây trùng có vòng đỏ, câu nêu lý do
`recordState`	Ghi nhận trạng thái mới	Ô giữ số j, S_i điền vào bảng S
`descend`	`try(i+1)`	Ô trống kế tiếp thành ô đang xét
`restoreState`	Trả lại trạng thái cũ	Ô xóa số, nền tím viền nét đứt
`solutionFound`	Xuất lời giải (bản dừng sớm)	Các ô thuật toán điền chuyển màu xanh ngọc
`noSolution`	`try(1)` kết thúc không lời giải	Báo đề vô nghiệm
`solutionListed`	Xuất lời giải (bản tìm hết)	Thêm lời giải vào bộ sưu tập, chạy tiếp
`searchExhausted`	`try(1)` kết thúc (bản tìm hết)	Báo tổng số lời giải

Ba cấu trúc nữa giữ phần minh họa chạy mượt:

- **Vòng phát lại theo khung hình.** Mỗi khung hình, khoảng 60 lần một giây, lấy một loạt sự kiện theo tốc độ người xem chọn, áp vào trạng thái, rồi mới vẽ lại một lần. Vẽ lại sau từng sự kiện thì ở tốc độ cao trình duyệt sẽ treo.
- **Lịch sử 100 bước gần nhất.** Website không lưu bản sao lưới ở mỗi bước, chỉ lưu sự kiện và vài giá trị đi kèm. Khi người xem chọn một bước cũ, website hoàn tác ngược từ trạng thái hiện tại về bước đó.
- **Bộ đếm** lần thử, ghi nhận, trả lại, cập nhật theo từng sự kiện. Trang So sánh và trang Mọi lời giải gọi hai bộ đếm sau bằng chữ dễ hiểu hơn: ghi nhận là "Lần điền số", trả lại là "Lần quay lui".

Generator làm bộ giải chậm đi nhiều. Mỗi lời gọi `try(i + 1)` là một generator lồng trong `try(i)`. Sự kiện phát ra ở tầng sâu nhất phải được chuyển lên qua từng tầng phía trên (lệnh `yield*` của JavaScript) mới tới giao diện, có lúc qua 60 tầng. Bản đệ quy thường không phải chuyển gì. Trên đề Arto Inkala, bản generator mất 596 ms, bản đệ quy thường của trang So sánh, đo cùng lần, mất 4,42 ms, chậm hơn khoảng 135 lần. Ở bốn đề đã đo, mức chậm đi nằm từ 63 đến 135 lần (Chương 8). Phần so sánh tốc độ ở trang So sánh vì thế dùng bản đệ quy thường, để chi phí của generator không che mất khác biệt giữa các cách giải.

Chương 5

Ví dụ minh họa các bước thực hiện

Ví dụ của chương này là đề số 13, tên "Ngắn, có quay lui" trong danh sách đề mẫu của website. Đề có 16 ô trống và đúng một lời giải. Thuật toán cần 79 lần thử, ghi nhận 19 lần và trả lại 3 lần: một lần ở giai đoạn 1, hai lần ở giai đoạn 2. Hai lần ở giai đoạn 2 hợp thành một chuỗi lùi dây chuyền qua hai ô. Toàn bộ 83 bước nằm trong Bảng 5.3 ở cuối chương.

5.1 Chọn đề và cách đọc bảng

Chương này cần một đề đủ ngắn để in trọn từng bước, mà vẫn có quay lui thật. Đề Arto Inkala cần 445.778 lần thử, quá dài. Đề "Dễ" chỉ cần 27 lần thử nhưng không quay lui lần nào. Claude viết script `checks/find_illustration_puzzle.py`: xóa ngẫu nhiên 10 đến 16 ô từ một lưới đầy, rồi giữ lại các đề thỏa đúng một lời giải, 50 đến 90 lần thử, 1 đến 3 lần quay lui, có đủ ba kiểu bị loại vì hàng, cột và khối. Trong 12 đề tìm được, Claude chọn đề này vì cả ba lần quay lui nằm gọn trong 35 lần thử đầu, ngay trên hàng 1.

5	**3**	**4**	**6**	**7**		**9**		
6	**7**			**9**	**5**	**3**	**4**	**8**
	9	**8**	**3**	**4**		**5**	**6**	**7**
8		**9**	**7**	**6**			**2**	**3**
4		**6**		**5**	**3**	**7**	**9**	**1**
7	**1**	**3**	**9**		**4**	**8**	**5**	**6**
9	**6**	**1**	**5**	**3**	**7**	**2**		**4**
2	**8**	**7**	**4**	**1**	**9**	**6**	**3**	**5**
	4	**5**		**8**	**6**	**1**	**7**	**9**

(a) Đề: 16 ô trống

5	**3**	**4**	**6**	**7**	8	**9**	1	2
6	**7**	2	1	**9**	**5**	**3**	**4**	**8**
1	**9**	**8**	**3**	**4**	2	**5**	**6**	**7**
8	5	**9**	**7**	**6**	1	4	**2**	**3**
4	2	**6**	8	**5**	**3**	**7**	**9**	**1**
7	**1**	**3**	**9**	2	**4**	**8**	**5**	**6**
9	**6**	**1**	**5**	**3**	**7**	**2**	8	**4**
2	**8**	**7**	**4**	**1**	**9**	**6**	**3**	**5**
3	**4**	**5**	2	**8**	**6**	**1**	**7**	**9**

(b) Lời giải: số in xanh là số thuật toán điền

Hình 5.1: Đề số 13 và lời giải

Bảng 5.1 đánh số 16 ô trống theo thứ tự hàng rồi cột. S_i là số điền vào ô trống thứ i.

Bảng 5.1: Vị trí của 16 ô trống

i	1	2	3	4	5	6	7	8
Ô	(1, 6)	(1, 8)	(1, 9)	(2, 3)	(2, 4)	(3, 1)	(3, 6)	(4, 2)
i	9	10	11	12	13	14	15	16
Ô	(4, 6)	(4, 7)	(5, 2)	(5, 4)	(6, 5)	(7, 8)	(9, 1)	(9, 4)

Trạng thái dữ liệu gồm lưới `grid`, vectơ S và ba mảng đánh dấu `rowUsed`, `columnUsed`, `boxUsed` (Chương 4). Mỗi dòng của Bảng 5.3 là một bước thuật toán:

- **Bấm**: số lần bấm nút Từng bước trên trang Giải Sudoku để tới đúng trạng thái đó. Mỗi lần bấm, trang đi một sự kiện, kể cả các sự kiện không có dòng riêng trong bảng như "ghi nhận" và "sang ô kế". Vì vậy sau mỗi dòng Chấp nhận, cột Bấm tăng 3 (chấp nhận, ghi nhận, sang ô kế). Sau dòng Bị loại hay Trả lại, cột Bấm tăng 1. Ở ô cuối cùng, lần bấm 118 là chấp nhận, lần bấm 119 là ghi nhận, lần bấm 120 là xuất lời giải.

- **Lần thử**, i, **Ô**, j: bộ đếm lần thử, ô trống thứ mấy, nằm ở đâu, và số đang thử. Dòng Trả lại không tính là một lần thử, nên giữ nguyên bộ đếm của dòng trước, và j ở dòng đó là số vừa bị xóa.

- **Thao tác** và **Lý do**: "Chấp nhận" là số j chấp nhận được, thuật toán ghi nhận rồi sang ô trống kế tiếp. "Bị loại" kèm hàng, cột hoặc khối bị trùng và ô đang giữ số đó. Số trùng ở nhiều chỗ thì bảng ghi chỗ đầu tiên theo thứ tự hàng, cột, khối, đúng thứ tự thuật toán

kiểm tra. “Trả lại” là trả lại trạng thái cũ vì ô phía sau đã hết số để thử. “Có lời giải” là dòng cuối, không thử thêm số nào, nên bộ đếm lần thử vẫn là 79.

- S **sau bước**: vectơ $S_1 \dots S_{16}$ sau khi bước đó xong, dấu chấm là thành phần chưa có giá trị.

Muốn kiểm lại trên website, mở trang Giải Sudoku, chọn “Đề 13 - Ngắn, có quay lui” ở ô Đề mẫu, rồi bấm Từng bước. Nút “Tới lần quay lui kế” đưa thẳng tới dòng Trả lại kế tiếp, nút “Lùi một bước” đi ngược lại. Bộ đếm trên trang cộng dồn từ đầu: 3 ghi nhận nghĩa là thuật toán đã ghi nhận 3 lần, không phải lưới đang có 3 số do thuật toán điền.

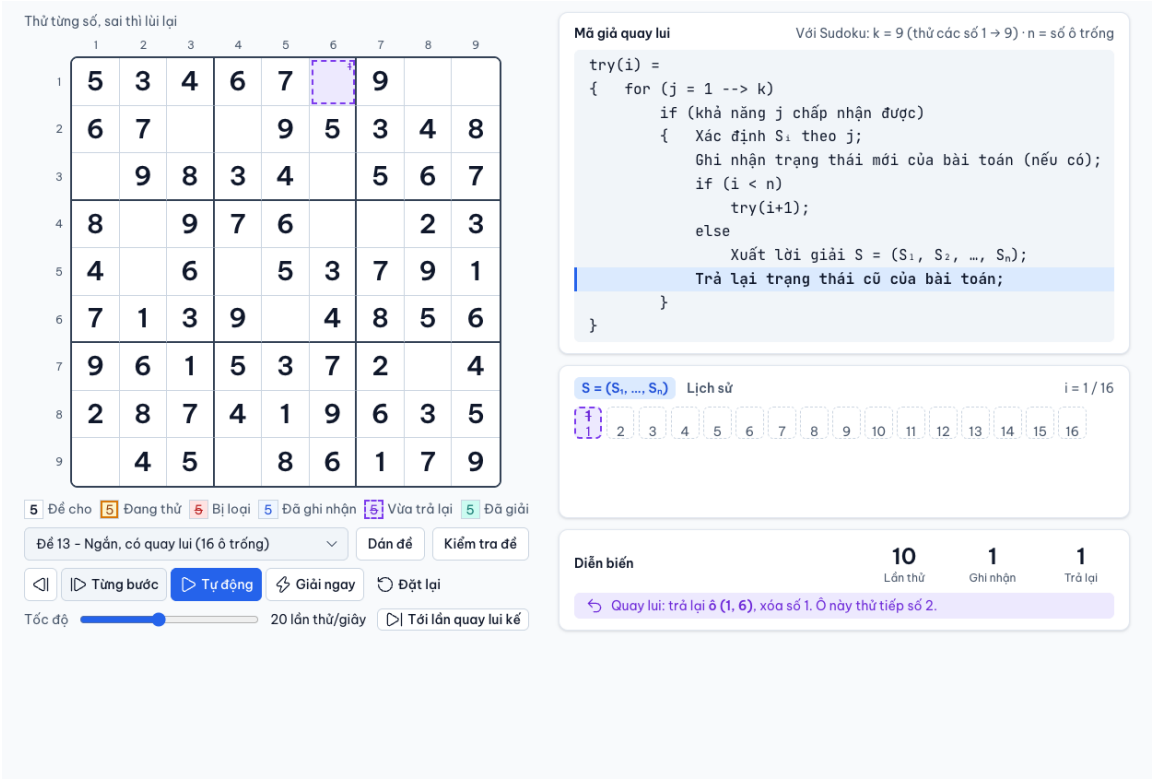
5.2 Diễn biến

Hàng 1 đã có các số 5, 3, 4, 6, 7, 9, nên ba ô trống $(1,6)$, $(1,8)$, $(1,9)$ phải nhận đúng ba số 1, 2, 8. Số 8 đã nằm ở ô $(2,9)$, cùng cột 9 với ô $(1,9)$ và cùng khối 3 với cả hai ô $(1,8)$, $(1,9)$. Vậy chỉ ô $(1,6)$ nhận được số 8. Người giải bằng tay thấy ngay điều này. Thuật toán thì thử từ nhỏ lên lớn, nên phải đi vào ngõ cụt hai lần mới tới số 8.

Giai đoạn 1: lần thử 1 đến 10, quay lui một ô

Ở lần thử 1, thuật toán thử số 1 ở ô $(1,6)$. Hàng 1, cột 6 và khối 2 đều chưa có số 1, nên số 1 chấp nhận được. Thuật toán ghi nhận: $S_1 = 1$, `grid[1][6] = 1`, bật `rowUsed[1][1]`, `columnUsed[6][1]`, `boxUsed[2][1]`. Rồi sang ô trống thứ 2.

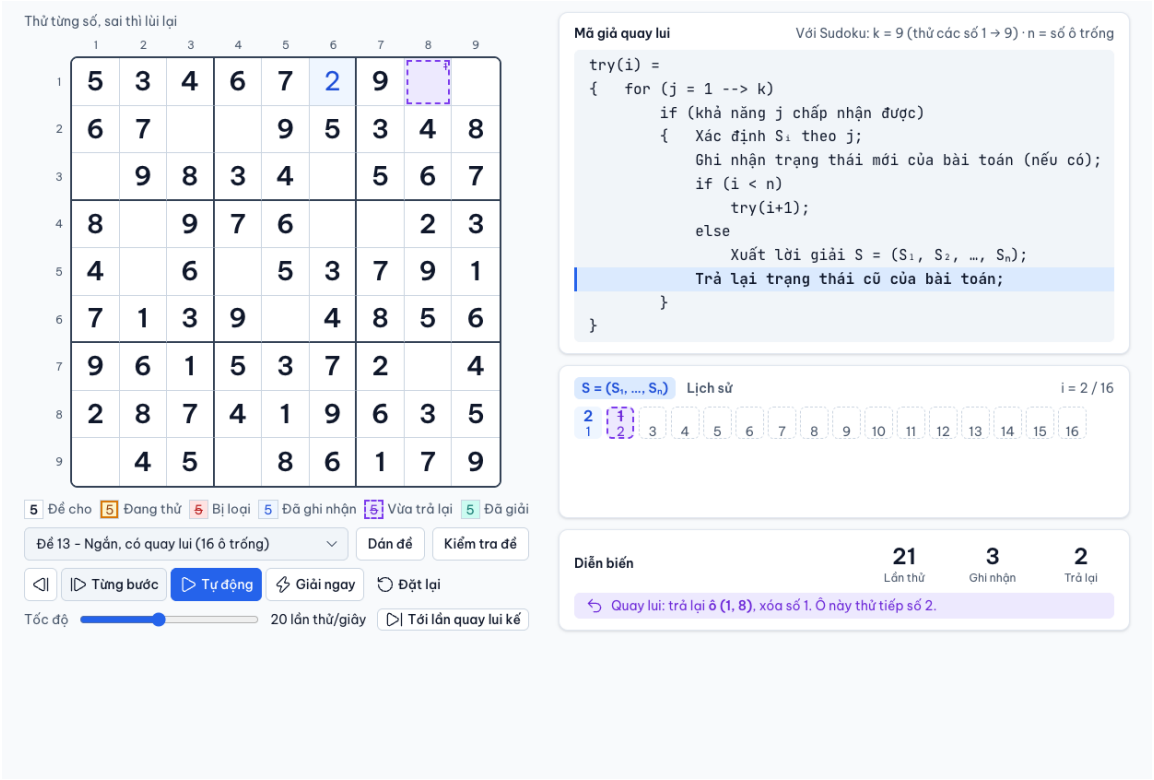
Ở ô $(1,8)$, cả 9 số đều trùng. Số 1 trùng ngay với số vừa điền ở ô $(1,6)$. Số 2 trùng cột 8, số 8 trùng khối 3, các số còn lại trùng hàng 1. Ô $(1,8)$ hết số, nên sau lần thử 10, ở lần bấm 13, thuật toán trả lại trạng thái cũ của ô $(1,6)$: xóa số 1 khỏi lưới, tắt ba cờ vừa bật, S_1 trở về trống (Hình 5.2).



Hình 5.2: Lần bấm 13: thuật toán vừa trả lại ô (1, 6), xóa số 1. Bộ đếm: 10 lần thử, 1 ghi nhận, 1 trả lại

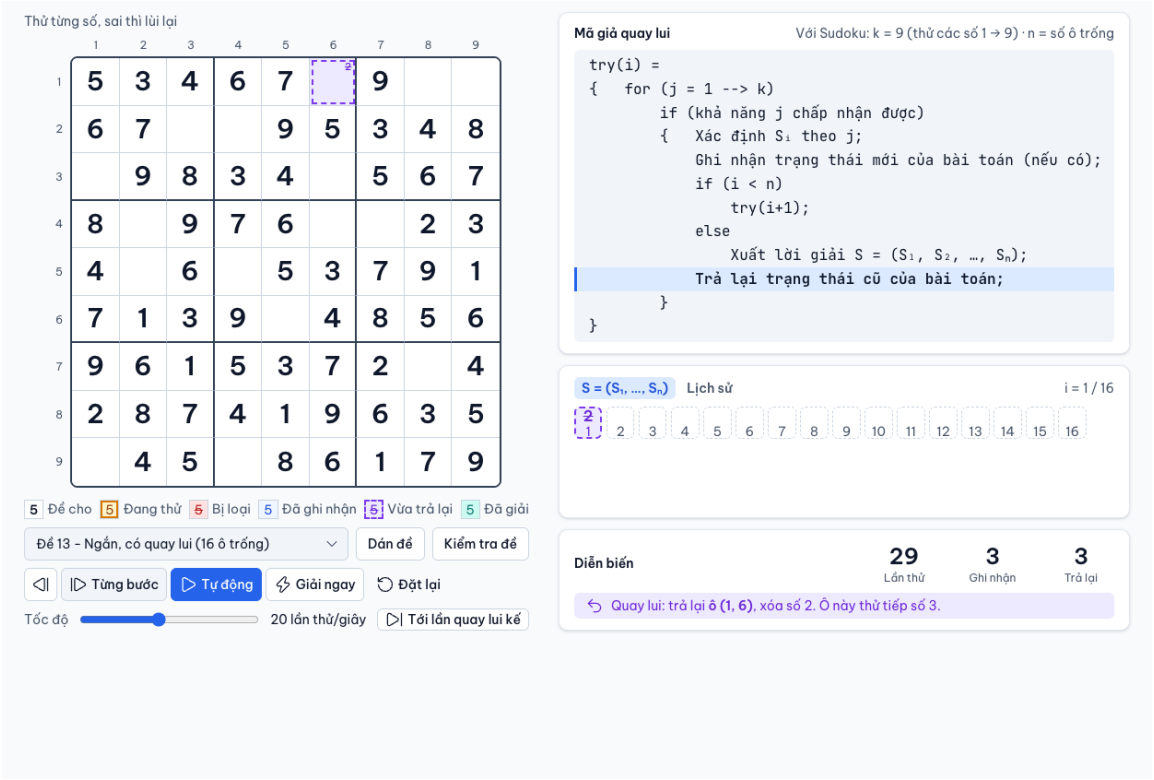
Giai đoạn 2: lần thử 11 đến 29, quay lui dây chuyền

Thuật toán thử tiếp số 2 ở ô $(1, 6)$, và số 2 chấp nhận được (lần thử 11). Lần này ô $(1, 8)$ nhận được số 1, vì số 1 không còn ở hàng 1 nữa (lần thử 12). Ở ô trống thứ 3, ô $(1, 9)$, cả 9 số đều trùng: số 1 trùng với ô $(1, 8)$ vừa điền, số 2 trùng với ô $(1, 6)$, số 8 trùng cột 9, các số còn lại trùng hàng 1. Sau lần thử 21, ở lần bấm 29, thuật toán trả lại ô $(1, 8)$, xóa số 1 (Hình 5.3).



Hình 5.3: Lần bấm 29: thuật toán vừa trả lại ô (1, 8), xóa số 1. Bộ đếm: 21 lần thử, 3 ghi nhận, 2 trả lại

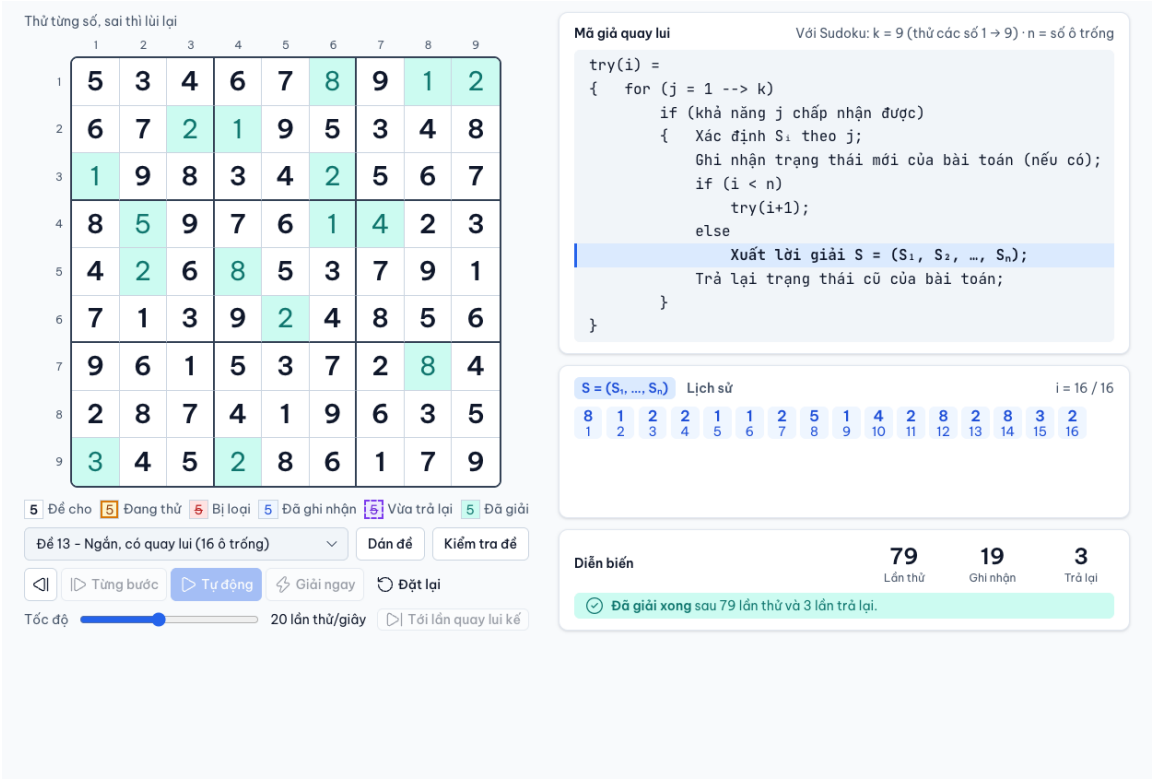
Ô $(1, 8)$ chưa hết số, nên thuật toán thử tiếp từ số 2. Số 2 trùng ô $(1, 6)$, số 8 trùng khối 3, các số còn lại trùng hàng 1. Ô $(1, 8)$ hết số lần thứ hai, nên sau lần thử 29, ở lần bấm 38, thuật toán lùi tiếp thêm một ô: trả lại ô $(1, 6)$, xóa số 2 (Hình 5.4). Hình này gần giống Hình 5.2. Chỗ khác là số bị gạch trong ô S_1, nay là số 2, và bộ đếm. Đây là quay lui dây chuyền: ô $(1, 9)$ hết số làm ô $(1, 8)$ phải đổi số, và ô $(1, 8)$ hết số làm ô $(1, 6)$ phải đổi số.



Hình 5.4: Lần bấm 38: thuật toán vừa trả lại ô (1, 6), xóa số 2. Bộ đếm: 29 lần thử, 3 ghi nhận, 3 trả lại

Giai đoạn 3: lần thử 30 đến 79, đi thẳng tới lời giải

Ở ô $(1, 6)$, các số 3 đến 7 đều trùng hàng 1. Số 8 chấp nhận được ở lần thử 35. Từ đây thuật toán không quay lui lần nào nữa. Ô $(1, 8)$ nhận số 1, ô $(1, 9)$ nhận số 2, rồi 13 ô còn lại lần lượt nhận số. Ở lần thử 79 (lần bấm 118), ô trống cuối cùng $(9, 4)$ nhận số 2, và ở lần bấm 120 thuật toán xuất lời giải (Hình 5.5).



Hình 5.5: Lần bấm 120: đã giải xong. Bộ đếm: 79 lần thử, 19 ghi nhận, 3 trả lại

Kết quả cuối cùng là lời giải ở Hình 5.1(b), với $S = (8, 1, 2, 2, 1, 1, 2, 5, 1, 4, 2, 8, 2, 8, 3, 2)$. Nếu không phải lùi lần nào, thuật toán chỉ cần thử $S_1 + S_2 + \cdots + S_{16} = 52$ lần, vì ô thứ i thử từ 1 tới S_i. Thực tế cần 79 lần, nên ba lần quay lui làm mất 27 lần thử, khoảng một phần ba tổng số. Thuật toán thử theo thứ tự cố định nên không "nhìn" trước như người giải bằng tay. Với cùng đề này, cách điền ô ít khả năng nhất trước (mục 4.5) chỉ cần 52 lần thử và không quay lui lần nào.

5.3 Ví dụ thứ hai: quay lui qua sáu ô

Ví dụ trên chỉ quay lui ở hàng 1. Đề mẫu số 3 "Vừa" có 40 ô trống, cần 1.909 lần thử và 187 lần trả lại, nên cho thấy một lần quay lui sâu hơn nhiều. Bảng 5.2 ghi các bước từ lần thử 783 đến lần thử 816, tức lần bấm 1.044 đến 1.089. Cột $S_{13} \ldots S_{19}$ chỉ giữ bảy thành phần của vectơ S đang thay đổi trong đoạn này, vì đề có tới 40 thành phần.

Ở lần thử 704, thuật toán điền số 8 vào ô $(4, 1)$, tức $S_{13} = 8$. Nhánh phía sau còn sống thêm 99 lần thử mới chết hẳn. Ở lần thử 783, ô $(5, 2)$ ($i = 19$) hết số: cả 9 số đều trùng. Từ đây thuật toán lùi liên tiếp qua sáu ô:

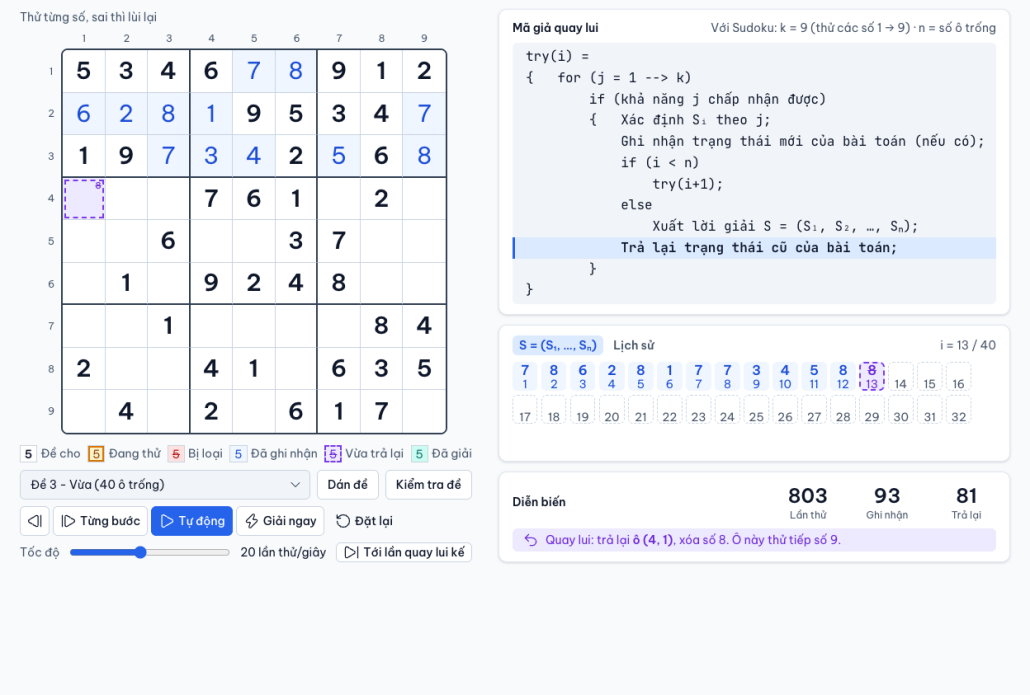
- Ô $(5, 1)$ đang giữ 4. Các số 5 đến 9 đều trùng (lần thử 784 đến 788), nên thuật toán trả lại ô này.
- Ô $(4, 9)$ đang giữ 3. Các số 4 đến 9 đều trùng (lần thử 789 đến 794), nên trả lại tiếp.
- Ô $(4, 7)$ đang giữ 4. Các số 5 đến 9 đều trùng (lần thử 795 đến 799), nên trả lại tiếp.

- Ô $(4, 3)$ đang giữ 9, số cuối cùng, nên không còn số nào để thử. Thuật toán trả lại ngay, không tốn lần thử nào.
- Ô $(4, 2)$ đang giữ 5. Các số 6 đến 9 đều trùng (lần thử 800 đến 803), nên trả lại tiếp.
- Ô $(4, 1)$ đang giữ 8, số điền ở lần thử 704. Ở lần thử 803, thuật toán trả lại ô $(4, 1)$ (Hình 5.7) và thử tiếp số 9, được chấp nhận ở lần thử 804.

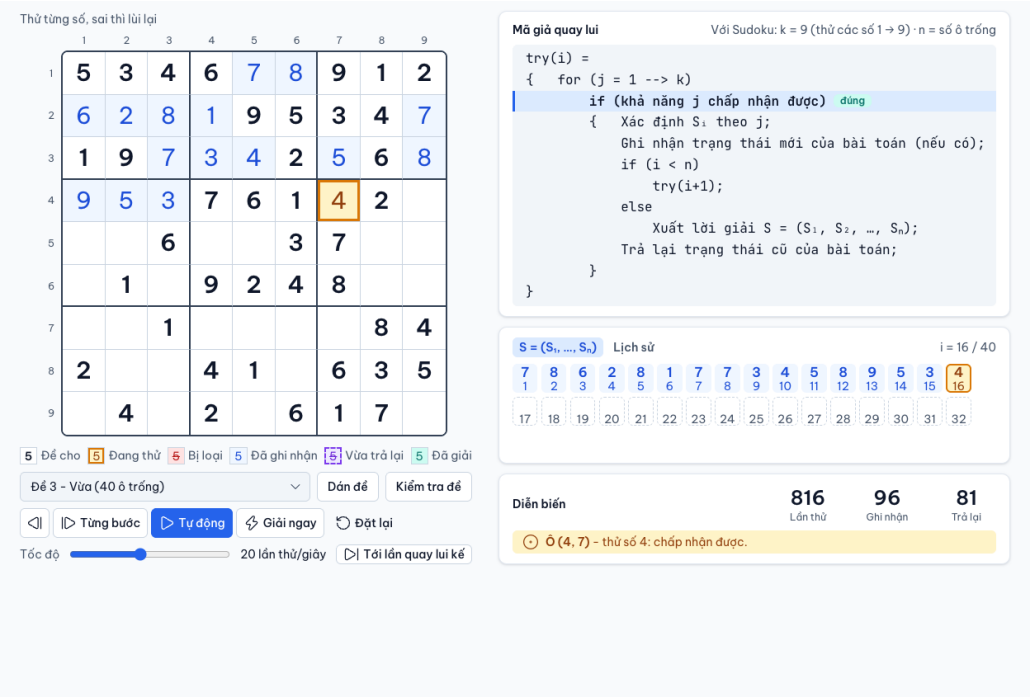
Sau đó thuật toán điền lại các ô phía sau. Ô $(4, 2)$ loại các số 1 đến 4 rồi nhận số 5 (lần thử 809). Ô $(4, 3)$ loại số 1 và 2 rồi nhận số 3 (lần thử 812). Ô $(4, 7)$ loại các số 1, 2 và 3 rồi nhận số 4 (lần thử 816). Số 8 mà ô $(4, 1)$ nhận ở lần thử 704 cũng nằm trong lời giải đầu tiên thuật toán tìm ra. Nhánh của nó chết vì trong các ô S_4 đến S_{12} phía trước còn ô mang số khác với lời giải đó ($S_4 = 2$ thay vì 7). Số 8 chỉ được điền lại ở lần thử 1.253, sau khi các ô đó đã đổi.



Hình 5.6: Lần bấm 1.044: ô $(5, 2)$ vừa hết số ở lần thử 783. Bộ đếm: 783 lần thử, 93 ghi nhận, 75 trả lại



Hình 5.7: Lần bấm 1.070: thuật toán vừa trả lại ô $(4, 1)$, xóa số 8. Bộ đếm: 803 lần thử, 93 ghi nhận, 81 trả lại



Hình 5.8: Lần bấm 1.089: ô $(4, 7)$ vừa nhận số 4 ở lần thử 816. Bộ đếm: 816 lần thử, 96 ghi nhận, 81 trả lại

Muốn kiểm lại trên website, chọn đề 3 “Vừa” ở trang Giải Sudoku rồi bấm Từng bước đủ số lần ở cột Bấm. Nút “Tới lần quay lui kế” cho nhảy qua từng lần trả lại thay cho việc bấm tay hàng nghìn lần.

Bảng 5.2: Các bước từ lần thử 783 đến lần thử 816 của đề "Vừa"

Bấm	**Lần thử**	i	**Ô**	j	**Thao tác**	**Lý do**	$S_{13} \ldots S_{19}$ **sau bước**
1044	783	19	(5, 2)	9	Bị loại	trùng cột 2 ở ô (3, 2)	8 5 9 4 3 4 .
1045	783	18	(5, 1)	4	Trả lại	ô sau hết số	8 5 9 4 3 . .
1046	784	18	(5, 1)	5	Bị loại	trùng cột 1 ở ô (1, 1)	8 5 9 4 3 . .
1047	785	18	(5, 1)	6	Bị loại	trùng hàng 5 ở ô (5, 3)	8 5 9 4 3 . .
1048	786	18	(5, 1)	7	Bị loại	trùng hàng 5 ở ô (5, 7)	8 5 9 4 3 . .
1049	787	18	(5, 1)	8	Bị loại	trùng cột 1 ở ô (4, 1)	8 5 9 4 3 . .
1050	788	18	(5, 1)	9	Bị loại	trùng khối 4 ở ô (4, 3)	8 5 9 4 3 . .
1051	788	17	(4, 9)	3	Trả lại	ô sau hết số	8 5 9 4 . . .
1052	789	17	(4, 9)	4	Bị loại	trùng hàng 4 ở ô (4, 7)	8 5 9 4 . . .
1053	790	17	(4, 9)	5	Bị loại	trùng hàng 4 ở ô (4, 2)	8 5 9 4 . . .
1054	791	17	(4, 9)	6	Bị loại	trùng hàng 4 ở ô (4, 5)	8 5 9 4 . . .
1055	792	17	(4, 9)	7	Bị loại	trùng hàng 4 ở ô (4, 4)	8 5 9 4 . . .
1056	793	17	(4, 9)	8	Bị loại	trùng hàng 4 ở ô (4, 1)	8 5 9 4 . . .
1057	794	17	(4, 9)	9	Bị loại	trùng hàng 4 ở ô (4, 3)	8 5 9 4 . . .
1058	794	16	(4, 7)	4	Trả lại	ô sau hết số	8 5 9 . . . .
1059	795	16	(4, 7)	5	Bị loại	trùng hàng 4 ở ô (4, 2)	8 5 9 . . . .
1060	796	16	(4, 7)	6	Bị loại	trùng hàng 4 ở ô (4, 5)	8 5 9 . . . .
1061	797	16	(4, 7)	7	Bị loại	trùng hàng 4 ở ô (4, 4)	8 5 9 . . . .
1062	798	16	(4, 7)	8	Bị loại	trùng hàng 4 ở ô (4, 1)	8 5 9 . . . .
1063	799	16	(4, 7)	9	Bị loại	trùng hàng 4 ở ô (4, 3)	8 5 9 . . . .
1064	799	15	(4, 3)	9	Trả lại	ô sau hết số	8 5 . . . . .
1065	799	14	(4, 2)	5	Trả lại	ô sau hết số	8 . . . . .
1066	800	14	(4, 2)	6	Bị loại	trùng hàng 4 ở ô (4, 5)	8 . . . . .
1067	801	14	(4, 2)	7	Bị loại	trùng hàng 4 ở ô (4, 4)	8 . . . . .
1068	802	14	(4, 2)	8	Bị loại	trùng hàng 4 ở ô (4, 1)	8 . . . . .
1069	803	14	(4, 2)	9	Bị loại	trùng cột 2 ở ô (3, 2)	8 . . . . .
1070	803	13	(4, 1)	8	Trả lại	ô sau hết số	. . . . .
1071	804	13	(4, 1)	9	Chấp nhận	không trùng	9 . . . . .
1074	805	14	(4, 2)	1	Bị loại	trùng hàng 4 ở ô (4, 6)	9 . . . . .
1075	806	14	(4, 2)	2	Bị loại	trùng hàng 4 ở ô (4, 8)	9 . . . . .
1076	807	14	(4, 2)	3	Bị loại	trùng cột 2 ở ô (1, 2)	9 . . . . .
1077	808	14	(4, 2)	4	Bị loại	trùng cột 2 ở ô (9, 2)	9 . . . . .
1078	809	14	(4, 2)	5	Chấp nhận	không trùng	9 5 . . . . .
1081	810	15	(4, 3)	1	Bị loại	trùng hàng 4 ở ô (4, 6)	9 5 . . . . .
1082	811	15	(4, 3)	2	Bị loại	trùng hàng 4 ở ô (4, 8)	9 5 . . . . .
1083	812	15	(4, 3)	3	Chấp nhận	không trùng	9 5 3 . . . .
1086	813	16	(4, 7)	1	Bị loại	trùng hàng 4 ở ô (4, 6)	9 5 3 . . . .
1087	814	16	(4, 7)	2	Bị loại	trùng hàng 4 ở ô (4, 8)	9 5 3 . . . .
1088	815	16	(4, 7)	3	Bị loại	trùng hàng 4 ở ô (4, 3)	9 5 3 . . . .
1089	816	16	(4, 7)	4	Chấp nhận	không trùng	9 5 3 4 . . .

5.4 Bảng toàn bộ các bước

Bảng 5.3: Toàn bộ các bước giải đề số 13

Bấm	Lần thử	i	Ô	j	Thao tác	Lý do	S sau bước
1	1	1	(1, 6)	1	Chấp nhận	không trùng	1 . . . . .
4	2	2	(1, 8)	1	Bị loại	trùng hàng 1 ở ô (1, 6)	1 . . . . .
5	3	2	(1, 8)	2	Bị loại	trùng cột 8 ở ô (4, 8)	1 . . . . .
6	4	2	(1, 8)	3	Bị loại	trùng hàng 1 ở ô (1, 2)	1 . . . . .
7	5	2	(1, 8)	4	Bị loại	trùng hàng 1 ở ô (1, 3)	1 . . . . .
8	6	2	(1, 8)	5	Bị loại	trùng hàng 1 ở ô (1, 1)	1 . . . . .
9	7	2	(1, 8)	6	Bị loại	trùng hàng 1 ở ô (1, 4)	1 . . . . .
10	8	2	(1, 8)	7	Bị loại	trùng hàng 1 ở ô (1, 5)	1 . . . . .
11	9	2	(1, 8)	8	Bị loại	trùng khối 3 ở ô (2, 9)	1 . . . . .
12	10	2	(1, 8)	9	Bị loại	trùng hàng 1 ở ô (1, 7)	1 . . . . .
13	10	1	(1, 6)	1	Trả lại	ô sau hết số	. . . . .
14	11	1	(1, 6)	2	Chấp nhận	không trùng	2 . . . . .
17	12	2	(1, 8)	1	Chấp nhận	không trùng	2 1 . . . . .
20	13	3	(1, 9)	1	Bị loại	trùng hàng 1 ở ô (1, 8)	2 1 . . . . .
21	14	3	(1, 9)	2	Bị loại	trùng hàng 1 ở ô (1, 6)	2 1 . . . . .
22	15	3	(1, 9)	3	Bị loại	trùng hàng 1 ở ô (1, 2)	2 1 . . . . .
23	16	3	(1, 9)	4	Bị loại	trùng hàng 1 ở ô (1, 3)	2 1 . . . . .
24	17	3	(1, 9)	5	Bị loại	trùng hàng 1 ở ô (1, 1)	2 1 . . . . .
25	18	3	(1, 9)	6	Bị loại	trùng hàng 1 ở ô (1, 4)	2 1 . . . . .
26	19	3	(1, 9)	7	Bị loại	trùng hàng 1 ở ô (1, 5)	2 1 . . . . .
27	20	3	(1, 9)	8	Bị loại	trùng cột 9 ở ô (2, 9)	2 1 . . . . .
28	21	3	(1, 9)	9	Bị loại	trùng hàng 1 ở ô (1, 7)	2 1 . . . . .
29	21	2	(1, 8)	1	Trả lại	ô sau hết số	2 . . . . .
30	22	2	(1, 8)	2	Bị loại	trùng hàng 1 ở ô (1, 6)	2 . . . . .
31	23	2	(1, 8)	3	Bị loại	trùng hàng 1 ở ô (1, 2)	2 . . . . .
32	24	2	(1, 8)	4	Bị loại	trùng hàng 1 ở ô (1, 3)	2 . . . . .
33	25	2	(1, 8)	5	Bị loại	trùng hàng 1 ở ô (1, 1)	2 . . . . .
34	26	2	(1, 8)	6	Bị loại	trùng hàng 1 ở ô (1, 4)	2 . . . . .
35	27	2	(1, 8)	7	Bị loại	trùng hàng 1 ở ô (1, 5)	2 . . . . .
36	28	2	(1, 8)	8	Bị loại	trùng khối 3 ở ô (2, 9)	2 . . . . .
37	29	2	(1, 8)	9	Bị loại	trùng hàng 1 ở ô (1, 7)	2 . . . . .
38	29	1	(1, 6)	2	Trả lại	ô sau hết số	. . . . .
39	30	1	(1, 6)	3	Bị loại	trùng hàng 1 ở ô (1, 2)	. . . . .
40	31	1	(1, 6)	4	Bị loại	trùng hàng 1 ở ô (1, 3)	. . . . .
41	32	1	(1, 6)	5	Bị loại	trùng hàng 1 ở ô (1, 1)	. . . . .
42	33	1	(1, 6)	6	Bị loại	trùng hàng 1 ở ô (1, 4)	. . . . .
43	34	1	(1, 6)	7	Bị loại	trùng hàng 1 ở ô (1, 5)	. . . . .
44	35	1	(1, 6)	8	Chấp nhận	không trùng	8 . . . . .
47	36	2	(1, 8)	1	Chấp nhận	không trùng	8 1 . . . . .
50	37	3	(1, 9)	1	Bị loại	trùng hàng 1 ở ô (1, 8)	8 1 . . . . .
51	38	3	(1, 9)	2	Chấp nhận	không trùng	8 1 2 . . . . .

Bấm	Lần thử	i	Ô	j	Thao tác	Lý do	S sau bước
54	39	4	(2, 3)	1	Bị loại	trùng cột 3 ở ô (7, 3)	8 1 2 . . . . .
55	40	4	(2, 3)	2	Chấp nhận	không trùng	8 1 2 2 . . . . .
58	41	5	(2, 4)	1	Chấp nhận	không trùng	8 1 2 2 1 . . . . .
61	42	6	(3, 1)	1	Chấp nhận	không trùng	8 1 2 2 1 1 . . . . .
64	43	7	(3, 6)	1	Bị loại	trùng hàng 3 ở ô (3, 1)	8 1 2 2 1 1 . . . . .
65	44	7	(3, 6)	2	Chấp nhận	không trùng	8 1 2 2 1 1 2 . . . . .
68	45	8	(4, 2)	1	Bị loại	trùng cột 2 ở ô (6, 2)	8 1 2 2 1 1 2 . . . . .
69	46	8	(4, 2)	2	Bị loại	trùng hàng 4 ở ô (4, 8)	8 1 2 2 1 1 2 . . . . .
70	47	8	(4, 2)	3	Bị loại	trùng hàng 4 ở ô (4, 9)	8 1 2 2 1 1 2 . . . . .
71	48	8	(4, 2)	4	Bị loại	trùng cột 2 ở ô (9, 2)	8 1 2 2 1 1 2 . . . . .
72	49	8	(4, 2)	5	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 . . . . .
75	50	9	(4, 6)	1	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 . . . . .
78	51	10	(4, 7)	1	Bị loại	trùng hàng 4 ở ô (4, 6)	8 1 2 2 1 1 2 5 1 . . . . .
79	52	10	(4, 7)	2	Bị loại	trùng hàng 4 ở ô (4, 8)	8 1 2 2 1 1 2 5 1 . . . . .
80	53	10	(4, 7)	3	Bị loại	trùng hàng 4 ở ô (4, 9)	8 1 2 2 1 1 2 5 1 . . . . .
81	54	10	(4, 7)	4	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 . . . . .
84	55	11	(5, 2)	1	Bị loại	trùng hàng 5 ở ô (5, 9)	8 1 2 2 1 1 2 5 1 4 . . . . .
85	56	11	(5, 2)	2	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 2 . . . . .
88	57	12	(5, 4)	1	Bị loại	trùng hàng 5 ở ô (5, 9)	8 1 2 2 1 1 2 5 1 4 2 . . . . .
89	58	12	(5, 4)	2	Bị loại	trùng hàng 5 ở ô (5, 2)	8 1 2 2 1 1 2 5 1 4 2 . . . . .
90	59	12	(5, 4)	3	Bị loại	trùng hàng 5 ở ô (5, 6)	8 1 2 2 1 1 2 5 1 4 2 . . . . .
91	60	12	(5, 4)	4	Bị loại	trùng hàng 5 ở ô (5, 1)	8 1 2 2 1 1 2 5 1 4 2 . . . . .
92	61	12	(5, 4)	5	Bị loại	trùng hàng 5 ở ô (5, 5)	8 1 2 2 1 1 2 5 1 4 2 . . . . .
93	62	12	(5, 4)	6	Bị loại	trùng hàng 5 ở ô (5, 3)	8 1 2 2 1 1 2 5 1 4 2 . . . . .
94	63	12	(5, 4)	7	Bị loại	trùng hàng 5 ở ô (5, 7)	8 1 2 2 1 1 2 5 1 4 2 . . . . .
95	64	12	(5, 4)	8	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 2 8 . . . .
98	65	13	(6, 5)	1	Bị loại	trùng hàng 6 ở ô (6, 2)	8 1 2 2 1 1 2 5 1 4 2 8 . . . .
99	66	13	(6, 5)	2	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 2 8 2 . . .
102	67	14	(7, 8)	1	Bị loại	trùng hàng 7 ở ô (7, 3)	8 1 2 2 1 1 2 5 1 4 2 8 2 . . .
103	68	14	(7, 8)	2	Bị loại	trùng hàng 7 ở ô (7, 7)	8 1 2 2 1 1 2 5 1 4 2 8 2 . . .
104	69	14	(7, 8)	3	Bị loại	trùng hàng 7 ở ô (7, 5)	8 1 2 2 1 1 2 5 1 4 2 8 2 . . .
105	70	14	(7, 8)	4	Bị loại	trùng hàng 7 ở ô (7, 9)	8 1 2 2 1 1 2 5 1 4 2 8 2 . . .
106	71	14	(7, 8)	5	Bị loại	trùng hàng 7 ở ô (7, 4)	8 1 2 2 1 1 2 5 1 4 2 8 2 . . .
107	72	14	(7, 8)	6	Bị loại	trùng hàng 7 ở ô (7, 2)	8 1 2 2 1 1 2 5 1 4 2 8 2 . . .
108	73	14	(7, 8)	7	Bị loại	trùng hàng 7 ở ô (7, 6)	8 1 2 2 1 1 2 5 1 4 2 8 2 . . .
109	74	14	(7, 8)	8	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 2 8 2 8 . .
112	75	15	(9, 1)	1	Bị loại	trùng hàng 9 ở ô (9, 7)	8 1 2 2 1 1 2 5 1 4 2 8 2 8 . .
113	76	15	(9, 1)	2	Bị loại	trùng cột 1 ở ô (8, 1)	8 1 2 2 1 1 2 5 1 4 2 8 2 8 . .
114	77	15	(9, 1)	3	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 2 8 2 8 3 .
117	78	16	(9, 4)	1	Bị loại	trùng hàng 9 ở ô (9, 7)	8 1 2 2 1 1 2 5 1 4 2 8 2 8 3 .
118	79	16	(9, 4)	2	Chấp nhận	không trùng	8 1 2 2 1 1 2 5 1 4 2 8 2 8 3 2
120	79				Có lời giải	đủ 16 ô	8 1 2 2 1 1 2 5 1 4 2 8 2 8 3 2

Chương 6

Quá trình thiết kế và xây dựng

Em dùng Claude Code để lập kế hoạch, viết code, viết test, đo đạc và viết nháp report. Em đặt yêu cầu, dùng thử, báo chỗ chưa ổn và chốt các lựa chọn. Phần lớn lựa chọn là phương án Claude đề xuất, chương này ghi rõ từng lần. Các giai đoạn kể theo thứ tự thời gian, từ ngày 3/10 đến ngày 13/10/2026.

Chương này dựa trên hai nguồn: nhật ký `work-log.md` và transcript của Claude Code lưu trên máy. Nhật ký do Claude ghi sau mỗi việc, em đọc lại. Khi soát report ngày 12/10, Hội đồng AI (AI council) đối chiếu nhật ký với transcript và tìm ra hai chỗ nhật ký ghi sai, đã sửa (mục 6.13). Prompt trong chương chép nguyên văn từ transcript, kể cả lỗi chính tả. Phần nào của tin nhắn bị lược thì thay bằng mô tả trong ngoặc vuông. Hai tin nhắn gửi cách nhau nằm ở hai khung riêng.

6.1 Công cụ và cách làm việc

Bảng 6.1 liệt kê các công cụ, thư viện và model em dùng.

Bảng 6.1: Công cụ, thư viện và model

Công cụ	**Vai trò**
Claude Code (tiện ích VS Code, phiên bản 2.1.287 đến 2.1.296), model Claude Opus 5.5, từ chiều 12/10 là Claude Sonnet 5.5	Lập kế hoạch, viết code website và các script, viết test, đo đạc, mở website trên Chrome để chụp màn hình và tự soát, ghi nhật ký, viết nháp chữ trên website và report
Các agent con do Claude Code gọi	Chấm ba tiêu đề trang chủ; Hội đồng AI năm vai đọc soát report
Một phiên Claude Code riêng	Viết lại 53 câu trên website cho tự nhiên hơn; đóng vai người dùng phổ thông, nhận xét website
Claude Design (trên claude.ai)	Vẽ 19 khung hình giao diện từ một prompt do Claude Code soạn; làm hoạt ảnh mở đầu, hoạt ảnh kết và nhạc nền của video demo
CapCut	Ghép video demo: đoạn mở đầu, đoạn quay màn hình, đoạn kết và nhạc nền, xuất 4K 25 khung hình/giây
Higgsfield	Thử tạo clip mở đầu và kết bằng AI, bỏ vì clip không phù hợp
Vercel, GitHub	Vercel chạy website tại địa chỉ công khai; GitHub lưu mã nguồn
Một công cụ AI dạng chat khác	Soạn ba bộ quy tắc văn phong và bài viết mẫu mà em dán vào Claude Code
React 19, React Router 8, TypeScript 6, Vite 8	Website chạy hoàn toàn trên trình duyệt, không có máy chủ
Vitest 5, Playwright 1.63, oxlint	Test đơn vị, test trên Chrome thật, soát lỗi code; Playwright cũng quay màn hình cho video demo
ffmpeg	Xử lý các đoạn quay màn hình trước khi ghép
Python 3.9	Các bản cài đặt tham chiếu để đối chiếu, script sinh bộ testcase
Yarn 4.18.1, lucide-react, Fontsource	Quản lý gói, biểu tượng, phông Be Vietnam Pro và JetBrains Mono
XeLaTeX	Report này

Mỗi việc đi qua cùng một vòng:

- Em nêu yêu cầu bằng một prompt ngắn.
- Claude đọc code hiện có, đo đạc khi cần, rồi đề xuất phương án. Chỗ nào có nhiều lựa chọn thì Claude hỏi lại em và ghi rõ phương án Claude khuyên.

- Em chọn. Claude làm, chạy test, mở website trên Chrome thật để chụp màn hình và tự soát.
- Em dùng thử, góp ý tiếp. Claude ghi việc vừa làm vào nhật ký.

Ngay prompt lập kế hoạch đầu tiên, em dặn Claude cách viết. Em giữ nguyên bảy dòng tiếng Anh:

Prompt · 3/10, kèm theo prompt lập kế hoạch

- Cut hype, buzzwords, empty phrasing and anything that reads like marketing copy.
- Write with a clear position. Fewer “it depends” hedges, fewer generic recommendations.
- Mix short, medium and long sentences. Uniform, too-perfect rhythm reads as machine-written.
- Replace vague claims with examples, scenarios and real detail — numbers, file paths, commit hashes.
- Use plain, natural words, as if explaining to someone without much background in the topic.
- Drop generic openers, filler, repeated transitions and summary paragraphs that add nothing.
- Match the user’s own voice: sample a few things they wrote, then follow that word choice, sentence rhythm and personality.

Bảy dòng này dặn Claude viết kế hoạch cho dễ đọc, có số liệu thật, bớt giọng quảng cáo. Về sau, chữ trên website và bản nháp report này cũng viết theo các luật đó, cộng với bộ quy tắc ở mục 6.11.

Luật về code em đặt ngay sau đó, và giữ suốt dự án:

Prompt · 3/10

Remember everything in code should be in English, code must be easy to read, easy to understand, must be explicit, can explain itself without comment. Do you think we can use React for the website?

6.2 Chọn đề và lập kế hoạch (3/10)

Em phân vân giữa Dijkstra và Sudoku.

Prompt · 3/10

Bạn nghĩ sao về Dijkstra, tôi cũng thích Sudoku

Claude khuyên bỏ Dijkstra vì trên lớp chưa học, cô chỉ nhắc tới khi cho xem sản phẩm của một sinh viên lớp chính quy. Claude khuyên chọn Sudoku: bài giải bằng quay lui, và có đủ ba động tác thử, loại, quay lui để nhìn thấy trên hình. Em đồng ý, và yêu cầu lập kế hoạch đầy đủ trước khi viết dòng code nào:

Prompt · 3/10

Ok, tôi đồng ý với Sudoku, nhưng chúng ta sẽ không thực hiện liền mà cần lên kế hoạch phân tích toàn bộ bài toán trước khi viết những dòng code đầu tiên. Vì vậy, hãy tạo 1 file markdown chứa toàn bộ chi tiết kế hoạch cho bài tập lớn này, lưu ý, giọng văn của bạn phải tuân thủ tuyệt đối theo yêu cầu của tôi dưới đây [bảy dòng luật văn phong ở mục trên]

Claude viết `plan.md` gồm 12 mục. Trước khi viết, Claude dựng một bản Python có bộ đếm số lần thử và chạy trên 7 đề để lấy số đo thật. Các quyết định thiết kế dựa vào số đo đó. Đề Arto Inkala cần 445.778 lần thử, nên không thể chỉ có hoạt ảnh chậm: website phải có thêm nút giải một mạch.

Prompt · 3/10

5.1. I wanna build a completely website and deploy to Netlify or any supported free server
6. UI must be nice, modern, can I use Claude Design for this?
[mục 12: 12.1 đến 12.4 đều "Agree", 12.5 "Wait for confirmation"]

Prompt · 3/10

Yes, file name should be in English as well. Don't need to care about zip and offline for now. Just let me know if we use React, does it affect to the performance?

Claude trả lời câu hỏi hiệu năng bằng số đo trên một bản thử viết nhanh. Trên đề Inkala, bản đệ quy thường chạy hết 3,7 ms. Bản viết thành generator, để giao diện dừng được ở từng bước, mất 623 ms. Phần chậm nằm ở generator, không nằm ở React. Từ số đo này, kế hoạch chốt cách vẽ: mỗi khung hình lấy một loạt bước rồi mới vẽ lại một lần, thay vì vẽ lại sau từng bước.

6.3 Thiết kế giao diện bằng Claude Design (3/10 - 4/10)

Prompt · 3/10

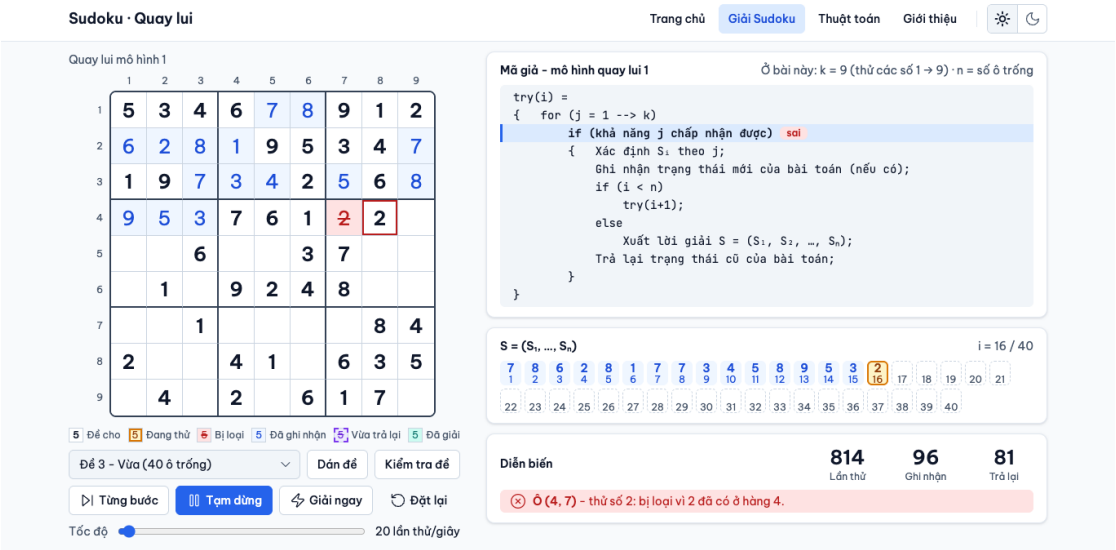
Can we build a very detail prompt then I can use to talk with Claude Design for the mockup? We should have the color pallet for the website, it should be mordern, no violate design rule.

Claude Code soạn file `design/claude-design-prompt.md`. File này có bốn điểm đáng giữ:

- Bảng màu có đo độ tương phản theo chuẩn WCAG AA. Một màu xám chỉ đạt 4,34:1, thiếu so với mức 4,5:1, nên Claude bỏ màu đó.
- Mỗi trạng thái ô có dấu hiệu ngoài màu, ví dụ số không hợp lệ có thêm nét gạch ngang, để người khó phân biệt đỏ và xanh vẫn đọc được.
- Trạng thái lưới lấy từ lần chạy thật của đề "Vừa" tại lần thử 803, lần thử 814 và lúc giải xong, không vẽ phỏng đoán.

- Mã giả chép đúng từng chữ slide mô hình 1 của cô.

Em đưa prompt vào Claude Design. Giữa chừng, em yêu cầu thêm nút chuyển giao diện Sáng, Tối. Claude Design trả về 19 khung hình, file màu `tokens.css` và hướng dẫn bàn giao. Hình 6.1 là một khung trong số đó.



Hình 6.1: Khung 04 của bản thiết kế: lần thử 814 của đề "Vừa", số 2 bị loại ở ô (4, 7)

Prompt · 4/10

Claude Design has finished their job and hand-off design in the folder design, you can take a look at this and let me know what your plan next?

Trước khi code theo bản thiết kế, Claude chạy lại thuật toán tới đúng thời điểm của từng khung để soát số. Khung 04 (lần thử 814), khung 05 (lần thử 803) và khung 17 (đề Inkala ở lần thử 200.000) đều khớp từng con số. Claude cũng chỉ ra bốn chỗ bản bàn giao lệch kế hoạch. Chỗ lớn nhất là cách chạy nút Giải ngay: kế hoạch cũ dùng bản đệ quy thường, nhưng bản này không dừng giữa chừng được. Claude đề xuất dùng một bộ giải duy nhất là generator cho cả ba nút Từng bước, Tự động và Giải ngay. Em đồng ý:

Prompt · 4/10

Ok, go ahead

6.4 Viết bộ giải bằng TypeScript (4/10)

Claude viết bộ giải bằng TypeScript. Bộ giải nằm trong thư mục `website/src/solver`, với hàm `tryStep(i)` giữ đúng các bước của mô hình 1. Khác biệt duy nhất: ở mỗi bước, hàm phát ra một sự kiện để giao diện vẽ. Bảng đối chiếu từng dòng nằm ở Chương 4.

Đáp án cho test lấy từ bản Python Claude dựng ngày 3/10, có ba bộ đếm: số lần thử, số lần ghi nhận, số lần trả lại. Hai bản phải cho cùng lời giải và cùng ba con số trên mọi đề. Lần đầu có 29 test, tất cả đạt.

Claude muốn biết test có bắt được lỗi thật không, nên thử làm sai thuật toán:

- Bỏ số 9 khỏi vòng thử: 10 test báo lỗi.
- Kiểm tra cột trước hàng: không test nào báo lỗi. Lý do: ba thời điểm chụp dùng làm đáp án đều chỉ trùng ở hàng. Claude thêm một test duyệt toàn bộ 1.682 lần bị loại của đề "Vừa", so cả lý do bị loại. Từ đó test bắt được lỗi này.

Giai đoạn này có hai sự cố nhỏ với trình quản lý gói:

Prompt · 4/10

Can we use yarn v2 instead of npm?

- Yarn 2 đã ngừng hỗ trợ, nên Claude dùng Yarn 4.18.1, cùng kiến trúc với bản 2. Lần chạy đầu, Yarn ghi nhầm ba file vào thư mục home vì ở đó có một file `yarn.lock` rỗng từ ngày 26/8. Claude xóa đúng ba thứ vừa tạo, giữ nguyên file cũ, rồi chạy lại trong thư mục dự án.
- Yarn 4 từ chối cài lucide-react 1.51.0 vì bản này phát hành chưa đủ 24 giờ. Claude giữ lớp bảo vệ đó và dùng bản 1.50.0.

6.5 Xây dựng trang Giải Sudoku (4/10)

Website lúc đầu còn ghi chữ "Wecode" trên giao diện. Em yêu cầu bỏ:

Prompt · 4/10

Yes, please go head. But before moving forward, I don't think label "Wecode" is ok, we can use another label, right?

Prompt · 4/10

I mean don't want any text relates to "Lab" "Wecode" like we're studying. Treat this website like the one to be used for people understand the algorithm, not my exercises

Từ đây, chữ trên website viết cho người muốn hiểu thuật toán. Mã giả vẫn chép đúng slide, chỉ tiêu đề đổi thành chữ chung như "Mã giả quay lui".

Trang Giải Sudoku có lưới gõ số được, chú thích màu, chọn đề mẫu, dán đề, các nút Từng bước, Tự động, Giải ngay, Đặt lại, thanh tốc độ, mã giả tô sáng dòng đang chạy, bảng $S = (S_1, \ldots, S_n)$, bộ đếm và câu thông báo.

Đề mẫu và đề chống quay lui

Em đề nghị thêm đề mẫu:

Prompt · 4/10

Go with A, and I think we should add more đề mẫu, what's ur suggestion?

Claude đề xuất năm đề, mỗi đề cho thấy một điều bảy đề cũ chưa có: đề nhiều lời giải, đề vô nghiệm phải tìm mới biết, AI Escargot, Easter Monster và đề chống quay lui. Đề chống quay lui cần 622.577.597 lần thử, bấm Giải ngay phải chờ nhiều phút. Em đề nghị tách đề này ra:

Prompt · 4/10

I love 12 but I don't think put it to our website is good. How about put 12 as a separated part to talk about there is a designed puzzle for "Anti-back tracking", and explain why. How is it?

Claude làm theo, thêm một mục riêng ở cuối trang Thuật toán. Mục này giải thích vì sao đề chậm bằng số đo: ở ô $(1, 1)$, sáu số sai đầu tiên phải đi hết nhánh của mình mới biết là sai, riêng sáu nhánh đó tốn 540.052.062 lần thử, tức 86,7 phần trăm tổng số.

Bảng lịch sử

Prompt · 4/10

I'm thinking about "Diễn biến" table should has history or last 10~15 steps, and can we trace back through history? Does it cost more performance?

Claude đo trên ba đề "Vừa", Wikipedia và Inkala: độ dài trung vị của một nhánh sai là 21 đến 31 bước. Giữ 100 bước thì thấy trọn khoảng 80 phần trăm số nhánh sai, giữ 15 bước chỉ thấy khoảng 30 phần trăm. Claude đề xuất giữ 100 bước, em đồng ý. Em yêu cầu thêm hai điều cho bảng này:

Prompt · 4/10

Every single column in table should have label to explain what they are

Prompt · 4/10

In history table, we should notice user that we show only last 100 records

Bảng lịch sử không tốn nhiều bộ nhớ. Website không lưu bản sao lưới ở mỗi bước, chỉ lưu sự kiện. Khi xem lại, website "hoàn tác" ngược từ trạng thái hiện tại. Test kiểm tra lùi từ 1 đến 300 sự kiện đều ra đúng trạng thái thật lúc đó.

Bố cục, chữ giao diện và test trên trình duyệt

Prompt · 4/10

[ba câu trả lời cho ba câu hỏi của Claude: “Yes can be”, “Let me think and get back later”, “Yes”] Screenshot: The solver screen, the components are very tight while we still have a lot of blank space below, can we make it fit the screen (responsive height)?

Prompt · 4/10

I don’t like put css in the same folder with tsx file. Any suggestion for folder structure?

Prompt · 4/10

And for the label, we should store them in a json file to preparing for localization/i18n in the future

Lưới giờ co giãn theo chiều cao màn hình, từ ô 44 px tới 96 px. CSS tách sang thư mục riêng. Khoảng 300 dòng chữ giao diện chuyển vào một file JSON. Claude kiểm tra không còn chữ tiếng Việt nào trong code ngoài file đó.

Claude viết một script điều khiển Chrome để bấm tới đúng lần thử của từng khung thiết kế rồi chụp màn hình so sánh. Cách soát này tìm ra các lỗi sau, tất cả đã sửa:

- Thanh trên cùng cao 57 px thay vì 56 px, làm trang dư 1 px.
- Đề sai thì bảng S biến mất.
- Đóng hộp dán đề thì con trỏ bàn phím không quay về nút “Dán đề”.
- 81 ô lưới vẽ lại ở mọi khung hình, vì hàm truyền xuống ô không cố định.
- Mục lục trang Thuật toán tô sai mục khi cuộn.
- Mở tab Lịch sử làm trang cao tới 2.998 px.

Sau đó Claude chuyển các bước soát này thành 31 test Playwright chạy trên Chrome thật. Để chắc bộ test có tác dụng, Claude đưa lại lỗi 57 px của ngày đầu. Ba test báo đúng “Expected 680, Received 681”.

6.6 Một phiên Claude riêng: viết lại chữ và đóng vai người dùng (4/10)

Em mở một phiên Claude Code riêng, không có ngữ cảnh của phiên chính. Prompt đầu nhờ phiên này sửa chữ trên website:

Prompt · 4/10, phiên riêng

You’re a specalist in language and how to speak naturally. You will review this vi.json file for each text and fix these label more naturally. A sentence must not be too short, ”cụt lủn”

Phiên đó viết lại 53 câu trong file chữ giao diện, sửa luôn đáp án của các test so từng ký tự câu thông báo, rồi chạy lại: 69 test đơn vị và 31 test trên Chrome đều đạt. Em chưa có người thật dùng thử, nên nhờ chính phiên này đóng vai người dùng phổ thông:

Prompt · 4/10, phiên riêng

As an end user (normal end user), how you rate my sudoku website? And feedback from your side for UI, for functional, any things else?

Em dán nguyên bản nhận xét sang phiên chính:

Prompt · 4/10

A feedback from an end-user [nguyên văn bản nhận xét]

Gần chín phút sau, em gửi thêm:

Prompt · 4/10

We can skip the video demo/source code for now. You can address the feedback from the end-user

Claude đo lại trước khi sửa. Bảng S ở màn hình 1366 × 768 cắt đôi hàng ô 33 đến 40, chữ chú thích màu chỉ cỡ 12 px. Hai điểm này đúng như nhận xét. Các thay đổi sau đó:

- Bảng S chỉ hiện trọn hàng, có thanh cuộn luôn thấy được.
- Rê chuột lên ô nào thì hiện nghĩa màu của ô đó. Chú thích màu lớn theo lưới.
- Thêm nút lùi một bước cạnh nút Từng bước, và nút tới lần quay lui kế tiếp.
- Chạy tự động thì dừng ở lần quay lui đầu tiên và hỏi người xem đoán bước tiếp theo.
- Trang chủ có lưới chạy lại thật các bước 783 đến 814 của đề “Vừa”, đứng yên nếu máy bật giảm chuyển động.

Claude không làm theo một góp ý. Nhận xét nói đề Wikipedia chạy tự động mất nửa giờ. Con số đó đúng ở tốc độ mặc định 20 lần thử mỗi giây, còn ở tốc độ tối đa 1.000 lần thử mỗi giây, đề này chạy hết 38 giây. Claude giữ tốc độ tối đa và thêm nút tới lần quay lui kế, để người xem nhảy thẳng tới chỗ thuật toán lùi lại.

Em cũng thấy một chữ Claude đặt nghe lạ và đề nghị đổi:

Prompt · 4/10

The word ”Người làm” is very weird, why you don’t use ”Tác giả” or more meaningful word?

Claude nhận đó là chữ của mình và đổi thành "Tác giả".

6.7 Yêu cầu chính thức và đổi góc nhìn (6/10)

Prompt · 6/10

Cô đã cập nhật yêu cầu cho final-project rồi, review qua xem chúng ta còn cần gì. Yêu cầu này sẽ overwrite yêu cầu ở buổi 8 [nguyên văn yêu cầu của cô]

Claude đối chiếu yêu cầu mới với kế hoạch và code: phần demo đã đủ, còn thiếu bộ testcase, report khoảng 10 mục, video đi qua mọi chức năng, và phần sáng tạo. Kế hoạch cũ ghi "không dùng thuật toán khác", trong khi tiêu chí mới chấm việc giải nhiều cách rồi so sánh. Claude đề xuất bốn phần mở rộng: so sánh ba cách kiểm tra, tìm mọi lời giải, chọn ô ít khả năng nhất trước, và lưới kích thước khác. Claude cũng tìm ra một lỗi trong kế hoạch: dòng "Dừng khi $k - i = 0$" phải là $n - i = 0$.

Prompt · 6/10

1. Đồng ý với gợi ý của bạn
2. Report bằng LaTeX
3. Sẽ quyết định sau khi chúng ta hoàn tất mọi thứ

Prompt · 6/10

Không nhất thiết cứ phải mention Lab 01 hay Wecode, hãy mở rộng phạm vi ra. Treat mọi thứ như chúng ta đang dựng 1 website để dạy học và chia sẻ kiến thức vậy

Claude hỏi lại "mở rộng" theo hướng nào: chỉ đổi góc nhìn, thêm trang bài học quay lui, hay thêm bài toán khác. Claude khuyên chỉ đổi góc nhìn và giữ Sudoku, vì 12 ngày không đủ cho bài toán thứ hai. Claude cũng khuyên làm cách chọn ô ít khả năng nhất trước (MRV) sau hai phần đầu. Em chọn đúng hai phương án Claude khuyên.

6.8 Bộ testcase (6/10)

Claude báo kế hoạch đã sửa xong và hỏi có làm bộ testcase luôn không. Em đáp:

Prompt · 6/10

Okay

Claude viết script sinh 294 đề chia 6 nhóm, đáp án lấy từ hai chương trình Python do Claude viết, tách khỏi code website. Chương 7 trình bày chi tiết. Mục này kể các vấn đề gặp phải.

Vòng lặp không dừng. Lần chạy đầu treo hơn 10 phút. Claude đo từng nhóm và tìm ra vòng sinh đề "đúng 2 lời giải" lặp mãi. Lý do: mọi lưới "ngẫu nhiên" đều sinh từ một lưới mẫu rồi

hoán vị hàng, cột và đổi tên số. Cả họ lưới đó không bao giờ có bốn ô hình chữ nhật đổi chỗ được. Lỗi này còn cho thấy bộ đề kém đa dạng hơn dự tính, nên Claude đổi sang sinh lưới đầy bằng quay lui với thứ tự số xáo trộn.

Bộ test lỏng. Mọi đề đạt ngay lần đầu. Claude kiểm lại bằng bốn lỗi cố ý, và lần đầu test chỉ bắt được hai lỗi:

- "Kiểm tra cột trước hàng" lọt qua, vì đáp án chỉ ghi lời giải và ba bộ đếm. Claude thêm ba cột: số lần bị loại vì hàng, vì cột, vì khối.
- "Bỏ qua khối khi kiểm tra đề" làm bộ giải chạy mãi trên đề trùng khối. Claude đặt giới hạn số lần thử cho mỗi đề, bằng đúng số trong đáp án.

Sau khi vá, test bắt được cả bốn lỗi, lần lượt 229, 5, 1 và 262 đề báo sai. Trong lúc thử, một lệnh chạy nền vẫn tiếp tục chèn lỗi vào code dù Claude đã ra lệnh dừng. Claude tắt hẳn tiến trình đó và khôi phục code từ git trước khi làm tiếp.

Prompt · 6/10

Wow, there are too much files .in and .out

294 đề nhân hai file là 588 file. Claude đề xuất gom mỗi nhóm thành một file văn bản, mỗi đề một khối gồm mô tả, đề và kết quả mong đợi. Em chọn cách đó.

Prompt · 6/10

Why it takes so long?

Hai đề chậm nhất chạy hết khoảng 19 phút trên bộ giải của website: đề chống quay lui cần 622.577.597 lần thử, đề 17 số cho sẵn cần 239.312.393 lần. Phần chậm nằm ở generator. Claude đề xuất giữ nguyên, vì bộ test phải chạy đúng bộ giải người xem thấy trên website, và cho hai đề này không chạy mặc định. Em chấp nhận cách đó.

6.9 Trang So sánh (6/10)

Prompt · 6/10

Ok, let's continue

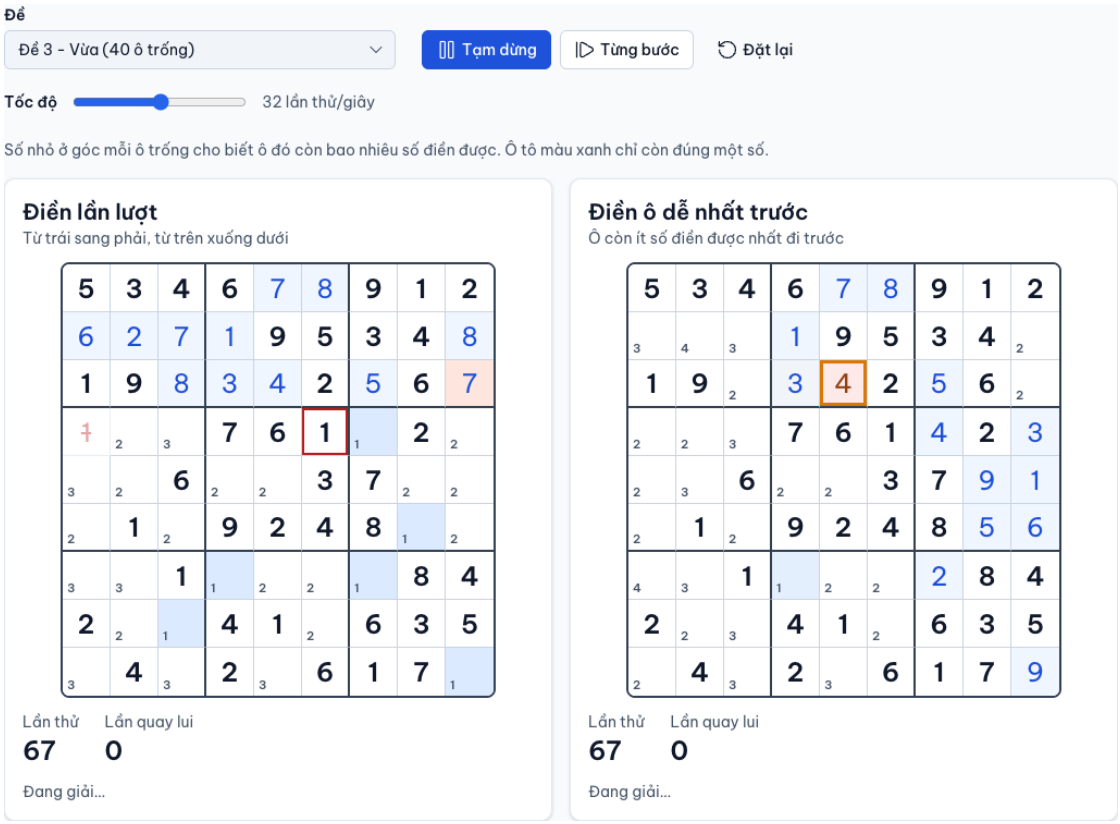
Claude viết phần thuật toán trước, đo xong mới làm giao diện. Ba cách kiểm tra một số có hợp lệ không (đọc lại lưới, ba mảng đánh dấu, bitmask) cho đúng cùng số lần thử trên mọi đề mẫu. Cách điền ô ít khả năng nhất trước giảm mạnh số lần thử: đề chống quay lui giảm từ 622.577.597 xuống 407.156. Trên đề Inkala, cách này lại chậm hơn bản thường, vì mỗi bước phải đếm lại khả năng của mọi ô trống (số đo ở Bảng 8.2).

Phiên bản đầu của trang có bảng số liệu và hai biểu đồ thanh. Em mở thử và không hiểu:

Prompt · 6/10, kèm ảnh chụp màn hình

Do we have any visualize for Comparing page? As a normal user, I don't understand any word of this page?

Hai biểu đồ để trống cho tới khi bấm Đo, và chữ toàn thuật ngữ. Claude đề xuất ba kiểu minh họa và khuyên cuộc đua giữa hai lưới. Em chọn cuộc đua (Hình 6.2): cùng đề, cùng tốc độ, lưới trái điền lần lượt, lưới phải điền ô dễ nhất trước, mỗi ô trống hiện số khả năng còn lại. Thuật ngữ đổi sang lời thường: "Nhìn lại lưới", "Sổ ghi chép", "Dãy công tắc".



Hình 6.2: Cuộc đua trên đề "Vừa" lúc đang chạy: số nhỏ ở góc mỗi ô trống là số khả năng còn lại

Để có đáp án riêng, Claude viết thêm một bản Python cho cách điền ô ít khả năng nhất trước. Bản này khớp bản TypeScript trên mọi đề mẫu.

6.10 Trang Mọi lời giải (6/10)

Mô hình 1 trên slide vốn không dừng ở lời giải đầu: xuất xong một lời giải, mô hình này trả lại trạng thái cũ rồi thử tiếp. Claude hỏi nên đặt chế độ "tìm hết" ở đâu. Lần này em không chọn phương án Claude khuyên, mà chọn đặt chế độ này ở một trang riêng, để trang Giải Sudoku giữ nguyên.

Khi chạy bản Python đếm lời giải, Claude phát hiện đề mẫu "Vừa", đề mặc định của trình giải, có 4 lời giải chứ không phải 1. Bộ đếm lời giải trong bộ testcase cũng ra 4. Claude sửa lại các chỗ tài liệu ghi sai điều này.

Claude kiểm chéo thêm: chế độ tìm hết đếm lại số lời giải của 251 đề trong bộ testcase. 239 đề tìm hết trong giới hạn 300 nghìn lần thử, và cả 239 đề khớp bộ đếm lời giải.

Chụp màn hình còn lộ thêm một lỗi: ở màn hình rộng 820 px, chữ trong menu sáu mục dồn xuống hai dòng. Claude đo thấy sáu mục vừa một dòng từ 880 px, nên cho menu thu gọn dưới 900 px.

6.11 Quy tắc văn phong và viết lại chữ trên website (6/10)

Em dán vào Claude Code ba bộ quy tắc văn phong và một bài viết mẫu. Cả bốn thứ này do một công cụ AI dạng chat khác soạn. Bộ đầu bắt đầu bằng:

Prompt · 6/10

Nguyên tắc dành cho bạn khi làm việc với câu chữ, hãy lưu vào system
• Quy tắc chống văn phong AI (Anti-AI Patterns): Cấm dùng các từ ngữ sáo rỗng, cầu kỳ quá mức (ví dụ: "trong bức tranh toàn cảnh", "tóm lại là", "không thể phủ nhận rằng"), ưu tiên câu ngắn, nhịp điệu linh hoạt.
• Quy chuẩn mạch lạc (Flow): Hướng dẫn cách chuyển ý tự nhiên giữa các đoạn, đi từ ví dụ thực tế rồi mới rút ra kết luận thay vì áp đặt lý thuyết khô khan

Claude chỉ ra ba chỗ các bộ quy tắc mâu thuẫn nhau hoặc mâu thuẫn với bài làm. Cấu trúc "câu chuyện trước" ngược với "kết luận trước". Luật "không đưa code" đụng tới mã giả cô chấm. Giọng hài hước không hợp với report. Claude đề xuất chia quy tắc theo nơi dùng: website dùng đủ cả bộ, report chỉ dùng luật về câu và bộ lọc từ.

Prompt · 6/10

Yeah, về mã giả thì bất khả xâm phạm, tôi đồng ý

Prompt · 6/10

Sao bạn không lưu vào như 1 SKILL?

Bộ quy tắc thành một skill của Claude Code, có bước tự kiểm trước khi giao chữ. Với bài viết mẫu, Claude chỉ ra hai lỗi thuật toán: "điền bừa một số" (thuật toán thử lần lượt từ 1 đến 9, không có gì bừa) và "thử hết tất cả các trường hợp" (thuật toán bỏ ngay số trùng, nên đề Inkala chỉ cần 445.778 lần thử thay vì 9^{60}).

Claude viết lại khoảng 50 câu trên website theo skill. Lần viết lại này để lọt một lỗi. Ba câu mới có chỗ chờ điền số, nhưng code chưa truyền số vào, nên trang hiện nguyên dấu ngoặc nhọn, và không test nào bắt được. Claude tự phát hiện ngay sau khi commit, sửa, rồi thêm một test mở

từng trang và kiểm tra không còn chỗ chờ điền nào hiện ra. Lần đầu chính test này vẫn đạt trên bản lỗi, vì test đọc chữ lúc trang còn đang tải. Claude đổi sang chờ trang tải xong, và test báo lỗi đúng ở trang Thuật toán.

Bảng 6.2 có ba câu trước và sau khi viết lại. Hai câu cuối còn được sửa thêm ngày 12/10, sau các lượt soát ở mục 6.13.

Bảng 6.2: Ba câu trên website trước và sau khi viết lại

Trước	**Sau**
Ô (4, 7) - thử số 2: bị loại vì 2 đã có ở hàng 4.	Ô (4, 7) - thử số 2: hàng 4 đã có số 2, bỏ.
Xem thuật toán quay lui giải Sudoku, từng bước một.	Thuật toán quay lui giải Sudoku như thế nào?
Nói cách khác, O(9^n) ở phần trên không chỉ là chuyện lý thuyết.	Đề này chưa chạm tới cận O(9^n) ở phần trên, vì 9^{64} là một số có 62 chữ số. Nhưng chỉ cần đáp án hàng đầu ngược với thứ tự thử, số lần thử đã lên hơn 600 triệu.

6.12 Đề minh họa và sự cố máy chậm (8/10 - 10/10)

Ngày 8/10, Claude tóm tắt việc còn lại và đề xuất làm đề minh họa cho Chương 5 trước. Em đồng ý:

Prompt · 8/10

totally agree

Claude viết script tìm đề theo tiêu chí: 10 đến 16 ô trống, đúng một lời giải, 50 đến 90 lần thử, 1 đến 3 lần quay lui, có đủ ba kiểu bị loại. Script ra 12 đề, trong đó Claude chọn đề 79 lần thử, 3 lần quay lui, vì cả ba lần quay lui nằm gọn trong 35 lần thử đầu trên hàng 1, và có một lần lùi dây chuyền qua hai ô. Đề này thành đề mẫu số 13 trên website. Một script khác xuất cả 83 bước bằng chính bộ giải của website, nên bảng ở Chương 5 khớp với demo.

Ngày 9/10, `yarn test` từ 9 giây thành 16 phút, mỗi lần chạy hỏng một test khác nhau vì quá giờ. Claude đo bằng một vòng lặp: cần 1,9 giây CPU mà mất 27 giây, tức máy chỉ cấp khoảng 5 đến 7 phần trăm sức chạy. Máy không bật tiết kiệm pin, cũng không có cảnh báo nhiệt. Claude nới giới hạn thời gian mỗi test lên 30 giây. Ngày 10/10 máy chạy bình thường trở lại, và toàn bộ test đạt: 723 test đơn vị, 292 testcase, 60 test trên Chrome.

6.13 Viết report và Hội đồng AI (12/10)

Claude đề xuất bắt đầu report, em đồng ý và để việc đưa website lên mạng lại sau:

Prompt · 12/10

Let's do it, I will deploy Netlify later

Claude viết Chương 6 trước, từ nhật ký và transcript. Giữa chừng, em đọc lại tiêu đề trang chủ mà Claude viết hôm 6/10, "Máy giải Sudoku cũng đoán sai. Xem nó lùi lại và sửa.", và thấy câu không giống người thật viết:

Prompt · 12/10

"Máy giải Sudoku cũng đoán sai. Xem nó lùi lại và sửa." Cái câu này đọc không giống người thường một chút nào cả, nghe là biết AI rồi. Bạn thử hỏi mấy con agent khác xem, nghe thấy gớm

Claude giao cho ba agent riêng, mỗi agent đóng một vai người đọc: người viết nội dung cho trang dạy code, sinh viên năm hai, biên tập viên tiếng Việt. Ba agent chấm ba tiêu đề: bản của Claude, bản em tự viết lại ("Cùng xem thuật toán quay lui giải Sudoku như thế nào nhé!") và bản gốc. Cả ba chấm bản của em thấp nhất, 3 đến 4 trên 10, vì "Cùng…nhé!" là giọng người dẫn chương trình. Bản của Claude được 6 đến 7, bản gốc 5 đến 7.

Cả ba agent cùng chỉ ra hai lỗi ở bản của Claude: chữ "nó" dịch từ "it" của tiếng Anh, và nhịp hai câu cụt kiểu khẩu hiệu. Biên tập viên còn bắt thêm lỗi trong đoạn dẫn: "mỗi cửa thử từng chìa" sai chủ ngữ, và hình ảnh dãy cửa chưa giải thích vì sao phải quay lại. Em chọn tiêu đề "Thuật toán quay lui giải Sudoku như thế nào?", là phương án agent người viết nội dung đề xuất, và dùng nguyên đoạn dẫn biên tập viên sửa. Các lỗi này thành luật mới trong skill văn phong.

Sau đó Claude viết Chương 1 đến 5, rồi Chương 7 và 8:

Prompt · 12/10

Okay, go ahead

Prompt · 12/10

Sound good, let's go with Ch.7 and Ch.8

Các bảng số trong report không gõ tay. Script `report/tools/build_report_tables.py` sinh bảng từ file dữ liệu, script `exportIllustrationTrace.ts` xuất 83 bước của đề 13 bằng chính bộ giải của website, và ảnh chụp lấy từ website thật bằng một script điều khiển Chrome.

Em muốn có người đọc soát report trước khi nộp:

Prompt · 12/10

Do you know how to set up agent councils to feedback our work?

Prompt · 12/10

Save as a skill first then we can start the council later

Prompt · 12/10

Start the council

Hội đồng AI (AI council) là một nhóm agent do Claude Code gọi song song trong cùng một lượt. Mỗi agent là một lần gọi model riêng, bắt đầu với ngữ cảnh trống, được giao một vai người đọc, và chỉ được đọc file, không được sửa. Các agent không thấy ý kiến của nhau. Claude Code giữ vai chủ tọa: gộp ý trùng, kiểm từng ý với file gốc rồi mới trình em. Em dùng cách này, thay vì nhờ Claude đọc lại bản nháp một lần, vì bốn lý do:

- Claude là bên viết bản nháp, nên khó thấy lỗi của chính mình. Agent mới không biết đoạn nào do AI viết.
- Mỗi vai bắt một loại lỗi khác: giảng viên nhìn mức đáp ứng yêu cầu, người soát thuật toán đối chiếu số liệu, biên tập viên bắt văn máy, người hoài nghi tìm câu phóng đại.
- Em chưa có người thật đọc thử bản nháp, và các vai này thay phần nào cho người đọc đó.
- Agent cũng có thể nhầm, nên không ý nào đến tay em khi chưa qua bước kiểm của chủ tọa.

Hội đồng AI không thay được người thật: các điểm do agent chấm chỉ là điểm thử.

Hội đồng AI gồm năm agent đọc độc lập, mỗi agent một vai: giảng viên chấm bài, người soát thuật toán, biên tập viên tiếng Việt, bạn cùng lớp và người hoài nghi. Năm agent nêu khoảng 130 ý. Claude gộp các ý trùng còn khoảng 45 ý, rồi tự kiểm từng ý với file gốc, dữ liệu và transcript trước khi trình em. Một số ý bị loại vì đi ngược khuôn của cô, ví dụ đề nghị viết $T(n) = \mathcal{O}(9^n)$ thay cho $\in \mathcal{O}(9^n)$, trong khi lời giải truy hồi cô chữa dùng dấu $\in$.

Các ý được giữ cho thấy bản nháp có những chỗ sai thật:

- Bản nháp ghi nhiều đề xuất của Claude thành "em chọn". Chương này đã ghi lại từng lần ai đề xuất, ai chọn.
- Bản nháp nói phiên Claude riêng ngày 4/10 không sửa file nào, trong khi phiên đó viết lại 53 câu trên website. Nhật ký cũng ghi sai tiêu đề trang chủ ngày 6/10.
- Câu "cận $\mathcal{O}(9^n)$ vẫn có thật" và câu trên website "đề này bắt thuật toán đi gần hết cái cây đó" nói quá: 622.577.597 lần thử vẫn còn cách xa 9^{64}. Câu đó được sửa cả trong report lẫn trên website.
- Mã giả bản Sudoku ghi "Xuất lời giải và dừng" mà không có cách dừng. Bản mới thêm biến `found`.
- Vài con số tính sai hoặc lệch nhau giữa các chương, ví dụ ba lần quay lui ở Chương 5 làm mất 27 lần thử chứ không phải 34. Hai đề chậm và tỉ lệ chậm của generator được đo lại trên code hiện tại.

Em chọn sửa hết các ý đã kiểm là đúng. Claude sửa report, sửa câu trên website, chạy lại test và các phép đo, rồi ghi vào nhật ký.

6.14 Đưa website lên Vercel và GitHub (12/10)

Kế hoạch ban đầu là Netlify. Ngày 12/10 em đổi sang Vercel và nói trước là sẽ đẩy mã nguồn lên GitHub cá nhân:

Prompt · 12/10

Deploy lên Vercel chứ k phải Netlify, tôi sẽ push commit lên git cá nhân trước

Prompt · 12/10

Commit và xóa nhánh backup. Git repo là 'https://github.com/honglax/solving-sudoku-by-backtracking.git'

Prompt · 12/10

Repo chỉ nên bao gồm trong thư mục website

Claude hỏi lại nên đẩy gì lên repo. Claude khuyên đẩy đủ thư mục `final-project`, để Vercel chọn thư mục `website` làm Root Directory, vì bộ test và báo cáo cần nằm cùng repo. Em chọn phương án đó, và repo để chế độ Private lúc đầu. Claude thêm file `website/vercel.json` với một luật chuyển mọi đường dẫn về trang `index.html`, để các trang con của website mở trực tiếp được từ địa chỉ gõ vào trình duyệt.

Lần push đầu qua HTTPS báo lỗi "Repository not found". Claude tìm ra nguyên nhân: Keychain của máy lưu sẵn một tài khoản GitHub khác, nên GitHub không nhận ra repo. Em chuyển sang SSH:

Prompt · 12/10

Ok, tôi đã dùng SSH, push complete

Em dựng dự án trên Vercel rồi gửi địa chỉ website:

Prompt · 12/10

Here it is: https://solving-sudoku-by-backtracking.vercel.app/

Tiếp theo em đọc README của repo và thấy thiếu hai thông tin:

Prompt · 12/10

Trang README thiếu thông tin tác giả cũng như mục đích của repo

Claude bổ sung hai phần đó vào README. Cô cần mở được mã nguồn từ report, nên trước hạn nộp em chuyển repo từ Private sang Public.

6.15 Video demo bằng Playwright (12/10)

Cô cho phép video không lời, chỉ cần chữ giải thích. Claude đề xuất hai cách quay: em tự quay màn hình bằng QuickTime, hoặc Claude dựng một script điều khiển Chrome. Claude khuyên script, vì video lặp lại được và không phụ thuộc giọng đọc. Em chọn script, và muốn thêm clip làm bằng AI:

Prompt · 12/10

1. Đồng ý
2. Đồng ý với Playwright option. Bên cạnh đó tôi muốn xây dựng một video bằng Claude kết hợp cùng Higgsfield

Claude viết bộ script trong `website/scripts/demoVideo/`. Một trang chủ chứa website trong khung 1366×680, đúng cỡ màn hình laptop mà thiết kế giao diện chọn, kèm một dải chữ giải thích bên dưới. Script bấm từng nút, vẽ khung sáng quanh thành phần đang giới thiệu và chạy con trỏ giả tới chỗ cần bấm, rồi Playwright thu thành video. `ffmpeg` ghép 10 đoạn thành một video. Nội dung 10 đoạn bám theo kịch bản trong kế hoạch: các trang chính, từng thành phần giao diện, ba cách nhập đề, chạy từng bước, chạy tự động, giải ngay, các đề đặc biệt, hai trang mở rộng, giao diện điện thoại và chạy bộ testcase. Đoạn cuối chạy thật lệnh `yarn testcases` rồi hiện lại kết quả, không gõ tay.

Em hỏi video có lên 4K được không, rồi nhắc lại ngay khi Claude đang trả lời:

Prompt · 12/10

You can try again with HiggFields. One more question. Can video has 4K resolution?

Prompt · 12/10

I mean, our recording should be 4K

Lần quay đầu với tỉ lệ điểm ảnh cao ra file đúng kích thước 3840×2160, nhưng nội dung chỉ chiếm góc trên trái, phần còn lại xám: Playwright không phóng khung hình. Bài thử ngắn đầu tiên không phát hiện vì chỉ cắt đúng vùng góc đó. Claude đổi cách làm: mở cửa sổ 3840×2160 thật, phóng toàn bộ trang chủ bằng CSS `zoom` 2,8125 để trình duyệt vẽ lại chữ ở độ phân giải gốc, và bấm nút bằng `element.click()` vì Playwright không bấm chuột được trong khung bị phóng. Claude kiểm bằng ảnh cắt 1:1 một khung hình: chữ sắc nét.

Với hai đoạn mở đầu và kết, Claude soạn hai prompt cho Higgsfield, một công cụ tạo video bằng AI. Cả hai kể chuyện một chùm chìa thử dãy ổ khóa. Đây là prompt mở đầu:

Prompt · 12/10, prompt do Claude Code soạn

A hand holds a ring of nine brass keys in front of a row of nine connected brass padlocks on a clean light studio background with soft blue accents. The hand tries keys on the padlocks one by one from left to right. At the fifth padlock every key fails and the lock stays shut. The hand moves back to the fourth padlock, swaps its key for the next one on the ring, and goes forward again. Smooth slow camera dolly along the row, shallow depth of field, no text, no numbers.

Việc tạo clip qua dòng lệnh bị Higgsfield từ chối vì tài khoản đang ở chế độ dùng thử, chỉ cho tạo trên website. Claude không đi vòng qua giới hạn đó. Em tạo thử trên website, xem kết quả và thấy clip xấu, không phù hợp, nên bỏ:

Prompt · 12/10

Bỏ qua idea video từ Higgfields vì video tạo ra không phù hợp, rất xấu. Dùng Claude Design tạo video được không?

Prompt · 12/10

Không cần dùng Playwright, thực thi với Claude Design đi rồi tôi sẽ mở Claude app để export ra video. Hoặc bạn hand over prompt để tôi tự làm

Claude làm theo cả hai cách em đề nghị. Claude soạn hai prompt trong file `video/claude-design-prompt.md`, và dựng sẵn hoạt ảnh bằng CSS trong một artifact Claude Design: một chùm chìa thử chín ổ khóa, ổ thứ năm kẹt nên chìa quay lại ổ thứ tư để đổi, rồi các ổ mở dần. Đây là đoạn đầu của prompt mở đầu:

Prompt · 12/10, prompt do Claude Code soạn

Design a 15-second animated intro, 1920×1080, 16:9, to open a screen-recorded demo video of a website that teaches backtracking through Sudoku. Animation only needs to play once and must be exportable as a 4K (3840×2160) video at 25 fps.
[bảng màu, phông chữ, ba dòng chữ ở đầu video, cảnh chìa khóa thử chín ổ khóa]

Em xuất hai đoạn mở đầu và kết từ Claude Design, và cũng dùng Claude Design làm nhạc nền. Ngoài ra Claude Code còn viết sẵn một bộ sinh nhạc nền ambient bằng code (`generateMusic.ts`) làm phương án dự phòng. Video cuối dùng nhạc từ Claude Design.

Về âm thanh, em dặn video không được câm, rồi nói rõ chỉ cần nhạc nền:

Prompt · 12/10

How about audio, the video should not be muted or without audio

Prompt · 12/10

Nah, tôi có thể thu âm, đừng dùng giọng người khác. Ý tôi là nhạc nền

Claude viết sẵn kịch bản thu âm (`video/narration-script.md`) và tùy chọn `--voice` cho bộ ghép, phòng khi em muốn tự thu lời đọc. Em còn yêu cầu thêm một bản chỉ có đoạn quay màn hình:

Prompt · 12/10

Tạo thêm cho mình một bản không có intro, outro và nhạc nền

Bản này dài 5 phút 28 giây, không có tiếng. Sau đó em báo sẽ tự ghép:

Prompt · 13/10

Tôi sẽ tự ghép video bằng Capcut, tiếp theo ta còn những bước nào

Gần 20 phút sau, em báo kết quả cuối của video, kèm ba việc tiếp theo cho Claude:

Prompt · 13/10

- Intro và outro được dựng từ Claude Design. Nhạc nền cũng sử dụng Claude Design. Ghép video bằng Capcut. Resolution 4K - 25FPS. Sản phẩm cuối cùng là sudoku-backtracking.mp4.
- Update report
- Khởi chạy council
- Làm 3 tùy chọn bạn gợi ý

Em ghép video bằng CapCut: đoạn mở đầu, đoạn quay màn hình do Playwright thu (có chữ giải thích, không có tiếng), đoạn kết và nhạc nền. Video xuất ở độ phân giải 4K, 25 khung hình mỗi giây, tên file `sudoku-backtracking.mp4`.

Đoạn quay màn hình được thu trước khi website có trang Kích thước lưới (13/10), nên video không có trang này. Lời giải thích ở cảnh menu điện thoại nói menu có sáu trang, nay website có bảy. Cảnh cuối chạy bộ testcase hiện 292 trên 292 đề, vì hai đề chậm nhất chạy riêng, còn bộ testcase có 294 đề. Trang Kích thước lưới xem được trực tiếp trên website.

Cách quay này làm lộ ra một lỗi thật mà 723 test đơn vị và 60 test trên Chrome đều bỏ sót. Khi giải xong đề vô nghiệm rồi đổi sang một đề khác, trang Giải Sudoku báo "Đã xảy ra lỗi". Claude tái hiện riêng và đọc thông báo trong console: "Digit 7 was rejected at (1, 5) but no unit holds it". Nguyên nhân nằm ở thứ tự cập nhật. Khi đề đổi, trang vẽ lại ngay với đề mới nhưng tiến trình cũ vẫn còn một lượt, nên câu thông báo "không có lời giải" của đề cũ đem đối chiếu với lưới của đề mới. Claude viết test trước, thấy test hỏng đúng như vậy, rồi sửa: bộ phát lại báo đề nào vừa được nạp, và trang bỏ qua tiến trình không thuộc đề đang hiện. Hai trang So sánh và Mọi lời giải được thử cùng kiểu, không bị lỗi này. Sau khi sửa có 723 test đơn vị và 62 test trên Chrome, đều đạt.

6.16 Ba phần bổ sung sau Hội đồng AI (13/10)

Sau lượt sửa theo Hội đồng AI, Claude gợi ý ba việc nếu còn thời gian: thêm một ví dụ minh họa có quay lui sâu hơn, thêm phân tích độ phức tạp trường hợp tốt nhất và của từng biến thể, và thêm lưới 4×4, 16×16. Claude nói thẳng là không khuyên làm việc thứ ba ở giai đoạn này vì tốn công nhất. Em vẫn chọn làm cả ba, bằng dòng cuối trong tin nhắn ở mục video: "Làm 3 tùy chọn bạn gợi ý".

Việc đầu là mục "Ví dụ thứ hai" ở Chương 5. Claude chọn đoạn từ lần thử 783 đến 816 của đề "Vừa" vì có một lần lùi liên tiếp qua sáu ô, xuất bảng bước từ chính bộ giải của website và chụp ba ảnh bằng script. Bộ đếm trên từng ảnh khớp bảng. Việc thứ hai là mục trường hợp tốt nhất và độ phức tạp các biến thể ở Chương 4.

Việc thứ ba mất nhiều công nhất. Bộ giải 9×9 của website dùng hằng số 9 ở khắp nơi, và 723 test đơn vị đang dựa vào nó. Claude chọn không sửa bộ giải đó. Claude viết một bộ giải riêng theo kích thước lưới trong thư mục `website/src/gridSizes`, một trang riêng cho nó, và để nguyên bộ 294 đề, vì video demo đã quay xong với con số đó. Trước khi viết code, Claude viết bản Python riêng cho các kích thước và dùng script ngẫu nhiên để tìm đề: sinh 40 đề cho mỗi mức số ô trống rồi chọn những đề có số lần thử vừa phải. Với 16×16, đề 120 ô trống cần từ 28.937 đến 1,5 triệu lần thử, còn đề 140 ô trống không đề nào giải xong trong 3 triệu lần thử, nên Claude dừng ở 120 ô.

Kiểm tra quan trọng nhất là so bộ giải mới với bộ giải 9×9 cũ: trên 8 đề, hai bộ phát ra đúng cùng chuỗi sự kiện, từng sự kiện một. Ngoài ra 11 đề đều khớp bản Python về số lần thử, số lần ghi nhận, số lần trả lại và lời giải đầu tiên.

Hai chỗ Claude làm chưa tốt, do em chỉ ra hoặc do kiểm tra tìm thấy:

- **Tên mục.** Claude đặt tên mục trên thanh điều hướng là "Cỡ khác". Em phản hồi:

Prompt · 13/10

"Cỡ khác" là gì? Một từ chẳng có ý nghĩa gì cả?

Claude đổi thành "Cỡ lưới". Em vẫn thấy chưa ổn:

Prompt · 13/10

Tại sao nhất thiết phải là 2 từ, nghe rất thô và không hiểu gì cả. Kích thước bảng hoặc kích thước lưới thì sao?

Tên cuối cùng là "Kích thước lưới".

- **Thanh điều hướng bị tràn.** Thêm mục thứ bảy làm các mục không còn vừa một dòng ở độ rộng dưới khoảng 1.010 px, thanh bị tràn ngang. Claude đo lại ở nhiều độ rộng, chuyển ngưỡng thu thanh điều hướng vào menu từ 900 lên 1.020 px và sửa hai test tương ứng.

Sau khi xong có 772 test đơn vị và 75 test trên Chrome, đều đạt. Lượt sửa theo Hội đồng AI lần hai thêm một test trên Chrome nữa (trang Giới thiệu dẫn tới mã nguồn), nên tổng cộng là 76.

6.17 Hội đồng AI lần hai trước khi nộp (13/10)

Sau ba phần bổ sung, Claude chạy Hội đồng AI lần hai ở chế độ soát cuối. Hội đồng AI có sáu agent: năm vai của lần đầu và một người kiểm thử giao diện, đọc ảnh chụp mọi trang ở hai cỡ màn hình, sáng và tối. Các agent nêu khoảng 115 ý. Claude gộp ý trùng rồi kiểm từng ý với file gốc, dữ liệu và transcript: khoảng 50 ý đúng, trong đó 10 ý nặng.

Các ý nặng cho thấy bản nháp vẫn còn lỗi thật dù đã qua Hội đồng AI lần đầu:

- Ví dụ thứ hai ở Chương 5 gọi số 8 ở ô $(4, 1)$ là sai. Số 8 đó nằm trong lời giải đầu tiên của đề. Lỗi nằm ở các ô điền trước nó.
- Chương 4 viết hai câu sai về biến thể tìm mọi lời giải: "số lần thử bằng số nút" (thực tế bằng 9 lần số nút) và "không còn trường hợp tốt nhất" (đề "Dễ" vẫn chỉ cần $9n$ lần thử).
- Chương 6 ghi một model và một phiên bản Claude Code. Transcript cho thấy Claude Opus 5.5 làm việc đến chiều 12/10, sau đó là Claude Sonnet 5.5. Chương 6 còn gán cho em vài việc Claude làm, và một câu "em đồng ý" mà transcript không có.
- Ba câu trên website nói quá về cận 9^n, như "đề 60 ô trống sẽ tốn cỡ 9^{60} lần thử". Cận chỉ cho biết số lần thử không thể vượt quá con số đó.
- Phụ lục A dẫn một repo GitHub đang ở chế độ Private, nên cô không mở được.

Claude cũng loại một số ý sai. Ví dụ, người kiểm thử giao diện nói màu trạng thái ở chế độ tối thiếu tương phản. Claude tính lại từ bảng màu: bốn trạng thái đạt từ 8,15 đến 10,11, vượt mức 4,5 của chuẩn WCAG AA.

Vai giảng viên chấm thử ba tiêu chí. Đây là điểm do agent chấm, không phải điểm của cô: mức đáp ứng 8,5 trên 10, mức đầu tư 9, độ sáng tạo 8,5. Ở lần đầu, các điểm tương ứng là 6, 8,5 và 8.

Em chọn sửa hết ý nặng, ý vừa, và các câu cụt trong ý nhẹ. Em cũng chọn chuyển repo sang Public, đóng gói bài nộp không kèm lịch sử git, và không quay thêm cảnh cho trang Kích thước lưới. Vì video đã quay xong, report ghi chú rõ điều này ở mục 6.15.

6.18 Phần việc của em và phần việc của AI

Bảng 6.3 tách phần em làm và phần Claude làm.

Bảng 6.3: Phân chia công việc

Việc	**Em**	**Claude**
Chọn đề, phạm vi, công nghệ	Chọn trong các phương án Claude đưa ra, thường theo lời Claude khuyên; một lần chọn khác (trang Mọi lời giải riêng)	Đề xuất phương án kèm lời khuyên, đo đạc để em chọn
Thuật toán	Đồng ý giữ nguyên mã giả của cô ở mọi nơi; đề nghị thêm đề mẫu và tách đề chống quay lui thành một mục riêng	Viết bộ giải TypeScript theo mô hình 1, các bản Python tham chiếu, ba cách kiểm tra, MRV và chế độ tìm hết
Giao diện	Duyệt bản thiết kế, thêm yêu cầu Sáng và Tối, chụp màn hình chỗ bố cục chật, yêu cầu nhãn cho từng cột	Claude Code soạn prompt; Claude Design vẽ; Claude Code code theo bản vẽ
Code website, script, test	Dùng thử và báo lỗi; đặt luật code tiếng Anh, tách CSS và chữ giao diện	Viết toàn bộ code, test, script đo đạc
Chữ trên website	Đưa bộ quy tắc văn phong; chỉ ra chữ "Người làm" và tiêu đề trang chủ nghe như máy viết	Viết chữ theo quy tắc, gọi agent chấm tiêu đề
Kiểm chứng	Dùng thử và hỏi lại khi thấy lạ (trang So sánh, thời gian chạy)	Đề xuất và chạy các cách soát: bản Python riêng, cố tình làm sai, chụp màn hình, Hội đồng AI
Report	Đặt yêu cầu, đọc bản nháp, chọn ý cần sửa sau Hội đồng AI, chịu trách nhiệm nội dung	Viết toàn bộ bản nháp từ nhật ký và transcript, sinh bảng và ảnh, sửa theo lựa chọn của em
Video demo	Chọn cách quay, bỏ Higgsfield, xuất hoạt ảnh mở đầu và kết từ Claude Design, làm nhạc nền bằng Claude Design, ghép bằng CapCut, xuất 4K	Viết script Playwright quay màn hình ở 4K, viết nội dung chữ giải thích, soạn prompt và dựng hoạt ảnh CSS cho Claude Design, tìm và sửa lỗi lộ ra khi quay

Phần em tự kiểm tra là dùng thử website như một người xem bình thường. Những lần dùng thử đó dẫn tới các thay đổi lớn: em không hiểu trang So sánh nên trang được làm lại thành cuộc đua hai lưới, em chụp màn hình trang Giải Sudoku bị chật nên lưới co giãn theo chiều cao, em thấy tiêu đề trang chủ giống máy viết nên có lượt chấm tiêu đề. Em cũng tự đổi tên gói của website thành "Sudoku - Backtracking".

Claude chạy các cách kiểm chứng sau xuyên suốt dự án:

- **Đáp án riêng.** Mọi số lần thử, ghi nhận, trả lại, số lần bị loại và số lời giải trên website và trong report đều so với các bản Python Claude viết riêng: bản tham chiếu mô hình 1, bản điền ô ít khả năng nhất trước, bản tìm hết, và bộ đếm lời giải dùng cách khác hẳn. Thời gian chạy thì không có bản nào để so, chỉ đo lại nhiều lần.
- **Cố tình làm sai.** Mỗi khi test đạt hết ngay lần đầu, Claude đưa lỗi vào thuật toán để xem test có bắt được không. Cách này hai lần làm lộ lỗ hổng trong bộ test.
- **Chụp màn hình trên Chrome thật** ở nhiều cỡ màn hình, sáng và tối, trên điện thoại.
- **Số đo thay cho phỏng đoán**, từ số bước giữ trong lịch sử tới chiều rộng menu.

Chương 7

Thiết kế thực nghiệm

Bộ testcase có 294 đề, chia 6 nhóm. Số ô trống trải từ 0 đến 81, số lần thử từ 0 đến 622.577.597. Đáp án của mọi đề lấy từ hai chương trình Python do Claude viết riêng, không lấy từ chính bộ giải đang được kiểm thử. Bảng 7.1 tóm tắt các nhóm.

Bảng 7.1: Các nhóm testcase và kết quả lần chạy ngày 12/10/2026, kể cả 2 đề chậm

Nhóm	Số đề	Ô trống	Lần thử	Đạt	Thời gian lớn nhất
Nhỏ	40	1 - 5	1 - 31	40/40	0,900 ms
Thông thường	150	20 - 50	91 - 43.531	150/150	29,4 ms
Lớn, khó	45	51 - 70	587 - 622.577.597	45/45	836.904 ms
Biên	13	0 - 81	0 - 3.195	13/13	3,30 ms
Đặc biệt	21	4 - 72	9 - 37.652	21/21	31,0 ms
Không hợp lệ	25			25/25	0,200 ms

7.1 Cách tạo bộ testcase

Script `testcases/tools/generate_testcases.py` sinh toàn bộ 294 đề với seed cố định 20261006. Chạy lại script sẽ ra đúng bộ đề cũ, không lệch ký tự nào. Script dùng năm cách tạo đề:

- **Lưới đầy ngẫu nhiên.** Quay lui trên lưới trống với thứ tự số xáo trộn, nên các lưới đầy trong bộ khác nhau từng đôi một. Bản đầu của script lấy một lưới mẫu rồi hoán vị hàng, cột và đổi tên số. Cách đó chỉ sinh được một họ lưới, nên Claude bỏ (Chương 6).
- **Đề có lời giải duy nhất.** Từ một lưới đầy, xóa lần lượt từng ô theo thứ tự ngẫu nhiên. Lần xóa nào làm đề có từ hai lời giải trở lên thì trả số lại. Dừng khi đủ số ô trống cần có.
- **Đề nhiều lời giải.** Xóa ngẫu nhiên nhiều ô, không kiểm tra lời giải duy nhất.

- **Đề dựng có chủ đích.** Đề đúng 2 lời giải: xóa 4 ô tạo thành hình chữ nhật mà hai số ở hai góc đối nhau đổi chỗ được. Đề vô nghiệm: lấy một đề có lời giải duy nhất, đổi một số cho sẵn thành số khác mà không tạo số trùng, rồi kiểm tra đề mới không còn lời giải nào. Đề không hợp lệ: thêm số trùng, ký tự lạ, bớt hoặc thêm dòng, ô.
- **Đề nổi tiếng.** Đề minh họa trên Wikipedia [5], đề Arto Inkala công bố năm 2012 [8], đề AI Escargot [9], đề Easter Monster [10], đề 17 số đầu tiên trong danh sách của Gordon Royle [6], và đề chống quay lui [4].

Đề sinh ngẫu nhiên nào cần hơn 1 triệu lần thử thì script bỏ và sinh đề khác, để bộ test chạy nhanh. Lựa chọn này làm các đề ngẫu nhiên nghiêng về phía dễ. Nhóm "Lớn, khó" vì thế có nhiều ô trống nhưng dễ hơn tên gọi của nó. Các đề khó thật sự là sáu đề nổi tiếng và đề chống quay lui.

Định dạng

Mỗi nhóm là một file văn bản. Mỗi đề trong file là một khối gồm dòng mô tả, phần `input` và phần `expected`. Phần input giữ nguyên từng ký tự, kể cả dòng trống, tab và ký tự xuống dòng kiểu Windows, dùng đúng các định dạng mà hộp Dán đề của website nhận. Phần expected là lưới lời giải, hoặc `NO SOLUTION`, hoặc `INVALID` kèm lý do. File `manifest.csv` ghi thêm cho mỗi đề: nhóm, định dạng, số ô trống, kết quả mong đợi, số lời giải, số lần thử, ghi nhận, trả lại, và số lần bị loại vì hàng, cột, khối.

7.2 Các nhóm testcase

Ngoài 294 đề của sáu nhóm dưới đây còn một file riêng cho lưới kích thước khác, nằm trong thư mục `testcases/other-sizes/` với tên `other-sizes.txt`. File có 11 đề: 5 đề 4×4, 2 đề 9×9 và 4 đề 16×16 (mục 4.6). Mỗi đề ghi lưới đầu vào, kết quả mong đợi, ba bộ đếm, số lời giải và lời giải đầu tiên. Đáp án lấy từ một chương trình Python viết riêng, `reference_sized_solver.py` trong thư mục `checks/`. Một test đơn vị đọc chính file này để so với bộ giải của website. File này tách khỏi 294 đề để số liệu ở Chương 8 không đổi.

- **Nhỏ (40 đề):** 1 đến 5 ô trống, người đọc tự kiểm bằng tay được. Gồm hai đề mẫu "Dễ nhất" và "Dễ".
- **Thông thường (150 đề):** 149 đề sinh ngẫu nhiên có lời giải duy nhất, 20 đến 50 ô trống, mỗi mức ô trống khoảng 5 đề. Cộng đề mẫu "Vừa" có 4 lời giải.
- **Lớn, khó (45 đề):** 25 đề có lời giải duy nhất với 51 đến 56 ô trống, 14 đề nhiều lời giải với 58 đến 70 ô trống, và 6 đề nổi tiếng.

- **Biên (13 đề):** lưới đã đầy (0 ô trống), lưới trống (81 ô), chỉ một số cho sẵn (80 ô), một ô trống ở ô đầu và ở ô cuối, trống trọn một hàng, một cột, một khối, đường chéo, ba hàng đầu, và ô trống duy nhất có đáp án là 1 hoặc là 9.
- **Đặc biệt (21 đề):** nhiều lời giải, đúng 2 lời giải, vô nghiệm ngay ô đầu, vô nghiệm phải tìm mới biết; 5 đề viết lại một đề ở nhóm khác theo định dạng 9 dòng liền hoặc chuỗi 81 ký tự; dòng trống xen giữa, xuống dòng kiểu Windows, nhiều dấu cách và tab.
- **Không hợp lệ (25 đề):** trùng ở hàng, cột, khối, nhiều chỗ cùng lúc; số 0, chữ cái, số 10, dấu chấm, dấu gạch; thiếu hoặc thừa dòng, ô; chuỗi 80 hoặc 82 ký tự; file rỗng.

Hai đề chậm

Đề chống quay lui cần 622.577.597 lần thử, đề 17 số cần 239.312.393 lần. Trên bộ giải của website, hai đề chạy hết 18 phút 42 giây (lần chạy ngày 12/10). Script đánh dấu chúng là đề chậm, không chạy mặc định. Lệnh `yarn testcases --include-slow` chạy cả hai.

7.3 Cách bảo đảm độ phủ

Khi chạy mỗi đề, script ghi lại những nhánh xử lý mà đề đó đi qua. Cả bộ phải đi qua đủ 20 nhánh, thiếu nhánh nào thì script báo lỗi:

- 3 định dạng đề: 9 dòng có dấu cách, 9 dòng viết liền, chuỗi 81 ký tự.
- 5 loại lỗi định dạng: sai số dòng, sai số ô trong một dòng, ký tự lạ trong dòng, chuỗi sai độ dài, ký tự lạ trong chuỗi.
- 3 loại số trùng: ở hàng, ở cột, ở khối.
- 5 kết quả: lưới đã đầy, giải được, giải được nhưng có nhiều lời giải, vô nghiệm ngay (chưa ghi nhận lần nào), vô nghiệm sau khi tìm.
- 3 lý do bị loại: vì hàng, vì cột, vì khối.
- Quay lui sau khi đã thử tới số 9.

Đi qua đủ 20 nhánh chưa chắc bộ test bắt được lỗi. Claude kiểm thêm bằng cách cố tình làm sai bộ giải của website ở bốn chỗ, rồi chạy lại cả bộ (Bảng 7.2). Lần đầu, bộ test chỉ bắt được hai trong bốn lỗi. Sau khi thêm ba cột số lần bị loại theo hàng, cột, khối, và đặt giới hạn số lần thử cho từng đề, bộ test bắt được cả bốn.

Bảng 7.2: Bốn lỗi cố ý và số đề báo sai

Lỗi cố ý	**Bộ test phát hiện**
Chỉ thử số 1 đến 8, bỏ số 9	229 đề sai, báo thiếu nhánh quay lui sau số 9
Kiểm tra đề bỏ qua khối 3×3	5 đề trùng khối báo sai, báo thiếu nhánh trùng ở khối
Nhận số 0 là ô trống	1 đề sai, đúng đề có số 0
Kiểm tra cột trước hàng	262 đề sai vì số lần bị loại theo hàng, cột lệch

7.4 Đáp án lấy từ đâu

Đáp án do hai chương trình Python tạo ra, tách hẳn khỏi code của website:

1. **Bản tham chiếu mô hình 1** (`checks/reference_solver.py`). Bản này cài đặt quay lui riêng, cùng thứ tự xét ô và thử số với website. Từ đây có lời giải đầu tiên, số lần thử, ghi nhận, trả lại và số lần bị loại theo hàng, cột, khối.

2. **Bộ đếm lời giải** trong script sinh đề. Bộ đếm dùng cách khác hẳn: chọn ô ít khả năng nhất trước và đánh dấu bằng bitmask. Bộ đếm cho biết đề có bao nhiêu lời giải, tối đa tới 1000.

Cả hai chương trình đều do Claude viết, cùng tác giả với bộ giải đang được kiểm. Nếu Claude hiểu sai một điểm của bài toán, cả hai bên có thể cùng sai. Bộ đếm lời giải bớt được rủi ro này vì đi theo cách giải khác hẳn, nhưng không loại hết được.

Script dừng ngay nếu hai chương trình không khớp nhau, hoặc nếu lời giải không đúng luật: 27 hàng, cột, khối phải là hoán vị của 1 đến 9, và các số đề cho phải giữ nguyên. Một kiểm tra chéo nữa đến từ trang Mọi lời giải. Chế độ tìm hết của website đếm lại số lời giải của 251 đề: bỏ 25 đề không hợp lệ, 2 lưới đã đầy, và 16 đề có từ 1000 lời giải trở lên mà bộ đếm chỉ ghi "1000+". 239 đề tìm hết trong giới hạn 300 nghìn lần thử, và cả 239 đề khớp bộ đếm độc lập. 12 đề còn lại vượt giới hạn đó nên không đối chiếu được bằng cách này.

Chương 8

Đánh giá

Lần chạy đầy đủ ngày 12/10/2026, kể cả hai đề chậm, cho cả 294 đề của bộ testcase đạt. Ngày 13/10/2026, 772 test đơn vị và 76 test trên Chrome cũng đạt. Các số đo trong chương chạy trên máy Apple M2 Pro, RAM 16 GB, macOS 26.7, Node 24.13 và Chrome 154. Số đo thời gian phụ thuộc máy, còn số lần thử thì không.

8.1 Kết quả trên bộ testcase

Lệnh `yarn testcases --include-slow` chạy cả 294 đề, kể cả 2 đề chậm, và cả 294 đề đạt (Bảng 7.1). Mỗi đề phải khớp đáp án ở bốn chỗ: loại kết quả, nội dung đầu ra, ba bộ đếm, và ba số lần bị loại theo hàng, cột, khối. Cả bộ đi qua đủ 20 nhánh xử lý.

Kết quả từng đề lưu trong `testcases/results/results.csv`. Lệnh `yarn testcases` không kèm 2 đề chậm chạy lại 292 đề còn lại trong khoảng 9 giây.

8.2 Thời gian thực hiện

- **Đề nhỏ và đề thông thường** chạy dưới 30 ms mỗi đề trên bộ giải của website.
- **Đề chậm nhất trong nhóm chạy mặc định** là Easter Monster: 2.357.880 lần thử, 3,37 giây.
- **Hai đề chậm:** đề chống quay lui mất 13 phút 57 giây, đề 17 số mất 4 phút 45 giây.

Phần lớn thời gian đó tốn vào việc minh họa từng bước. Bộ giải của website là một generator, và mỗi sự kiện phải đi qua từng tầng đệ quy mới tới giao diện (Chương 4). Trên đề Arto Inkala, bản generator mất 596 ms, bản đệ quy thường của trang So sánh mất 4,42 ms, chậm hơn khoảng 135 lần. Phép đo này chạy riêng với bảng so sánh ở mục sau, và cùng bản đệ quy đó trong bảng mất 5,17 ms, lệch khoảng 15 phần trăm, nên các tỉ lệ chỉ nên đọc là gần đúng. Trên ba đề mẫu khác, tỉ lệ này từ 63 đến 104 lần (`report/data/generator-overhead.csv`).

Trên Node, bộ giải của website chạy khoảng 744 nghìn lần thử mỗi giây. Nút Giải ngay trên Chrome giải đề Inkala trong khoảng 0,6 giây, tức khoảng 770 nghìn lần thử mỗi giây. Người xem bấm Dừng được bất cứ lúc nào, trang không treo.

8.3 So sánh các cách giải

Trang So sánh đo 6 cách: 2 thứ tự chọn ô nhân với 3 cách kiểm tra. Sáu đề đo là sáu đề mẫu cần nhiều lần thử nhất, không tính lưới trống vì lưới trống không phải một đề thật. Bảng 8.1 và Bảng 8.2 là kết quả chạy bằng script `report/tools/measureComparison.ts` ngày 12/10. Script dùng đúng hàm đo của trang So sánh: mỗi cách chạy lặp lại tới khi đủ 0,3 giây hoặc đủ 10.000 lần, rồi lấy trung bình.

Bảng 8.1: Số lần thử theo thứ tự chọn ô (ba cách kiểm tra cho cùng một số)

Đề	**Ô trống**	**Điền lần lượt**	**Ô dễ nhất trước**	**Giảm**
Vừa	40	1.909	226	8 lần
Wikipedia	51	37.652	239	158 lần
Arto Inkala	60	445.778	90.665	5 lần
AI Escargot	58	80.491	1.723	47 lần
Easter Monster	60	2.357.880	42.081	56 lần
Chống quay lui	64	622.577.597	407.156	1.529 lần

Bảng 8.2: Thời gian mỗi lần giải, tính bằng mili giây. Lưới: quét lại lưới; Sổ: ba mảng đánh dấu; Công tắc: bitmask

	Điền lần lượt			**Ô dễ nhất trước**		
Đề	Lưới	Sổ	Công tắc	Lưới	Sổ	Công tắc
Vừa	0,034	0,021	0,021	0,144	0,046	0,045
Wikipedia	0,750	0,396	0,381	0,275	0,078	0,070
Arto Inkala	9,55	5,17	4,67	81,9	23,5	20,5
AI Escargot	1,76	0,961	0,848	1,45	0,434	0,373
Easter Monster	53,3	29,3	24,6	33,9	10,6	8,74
Chống quay lui	17.632	8.588	8.892	767	170	167

Cấu trúc dữ liệu chỉ đổi hằng số. Ba cách kiểm tra cho đúng cùng số lần thử trên mọi đề, đúng như phân tích ở Chương 4: cách kiểm tra chỉ đổi c_2 trong $T(n) = 9\,T(n-1) + c_2$. Quét lưới chậm khoảng gấp đôi ba mảng đánh dấu, ví dụ 9,55 ms so với 5,17 ms trên đề Inkala. Bitmask nhanh hơn ba mảng đánh dấu khoảng 4 đến 16 phần trăm ở bốn đề, bằng nhau ở đề Vừa, và chậm hơn ở đề chống quay lui: 8.892 ms so với 8.588 ms.

Thứ tự chọn ô thì đổi cả cây tìm kiếm. Cách chọn ô ít khả năng nhất trước (MRV, trên website gọi là "Điền ô dễ nhất trước") cần ít lần thử hơn ở cả sáu đề. Mức giảm ít nhất ở đề Inkala, khoảng 5 lần (445.778 xuống 90.665), nhiều nhất ở đề chống quay lui, khoảng 1.529 lần (622.577.597 xuống 407.156).

Ít lần thử hơn chưa chắc nhanh hơn. Trước mỗi bước, MRV phải đếm lại khả năng của mọi ô trống còn lại. Ở đề Vừa và đề Inkala, số lần thử giảm chưa đủ bù phần đếm đó, nên tổng thời gian dài hơn: đề Inkala 23,5 ms so với 5,17 ms. Ở bốn đề còn lại, số lần thử giảm từ khoảng 47 lần trở lên và cách này nhanh hơn rõ: đề chống quay lui từ 8,6 giây xuống 0,17 giây, đề Easter Monster từ 29,3 ms xuống 10,6 ms.

8.4 Kiểm thử tự động

- **772 test đơn vị** (Vitest):
  - bộ giải khớp bản Python đến từng con số trên các đề mẫu;
  - trạng thái lưới khớp các khung của bản thiết kế;
  - ba cách kiểm tra và MRV khớp bản Python riêng của chúng;
  - chế độ tìm hết khớp số lời giải;
  - lịch sử lùi từ 1 đến 300 sự kiện vẫn ra đúng trạng thái;
  - câu thông báo đúng từng ký tự;
  - toàn bộ 292 đề nhanh của bộ testcase;
  - bộ giải theo kích thước lưới khớp bản Python trên 11 đề và phát ra đúng chuỗi sự kiện của bộ giải 9×9 trên 8 đề.
- **76 test trên Chrome thật** (Playwright): mọi trạng thái trong bản thiết kế, các nút Từng bước, Tự động, Giải ngay, Dừng, lịch sử, cuộc đua và phép đo ở trang So sánh, trang Mọi lời giải, bố cục ở nhiều cỡ màn hình từ 320 đến 2560 px, và không trang nào hiện chỗ chờ điền chưa thay số.
- **Soát code:** TypeScript ở chế độ nghiêm ngặt, oxlint, không còn chữ tiếng Việt nào trong code ngoài file ngôn ngữ.

Claude kiểm lại chính bộ test bằng các lỗi cố ý (Bảng 7.2). Lần đầu, bộ test để lọt hai trong bốn lỗi. Claude vá cả hai lỗ hổng, và bộ test sau đó bắt được cả bốn.

8.5 Tính trực quan của giao diện

Phần minh họa cho người xem thấy cùng lúc bốn thứ: lưới với màu riêng cho từng trạng thái ô, dòng mã giả đang chạy kèm nhãn đúng hoặc sai, vectơ S, và một câu giải thích bằng lời, ví dụ "Ô (4, 7) - thử số 2: hàng 4 đã có số 2, bỏ". Các chức năng hỗ trợ người học:

- Từng bước, lùi một bước, tự động với tốc độ từ 1 đến 1.000 lần thử mỗi giây, tới lần quay lui kế tiếp, giải ngay.
- Chạy tự động thì dừng ở lần quay lui đầu tiên và hỏi người xem đoán bước tiếp theo.
- Bảng lịch sử 100 bước gần nhất, bấm vào một dòng để xem lại đúng trạng thái lúc đó.
- Cuộc đua hai lưới ở trang So sánh, mỗi ô trống hiện số khả năng còn lại.
- Bộ sưu tập lời giải ở trang Mọi lời giải, tô các ô khác lời giải đầu tiên.
- Trang Kích thước lưới chạy cùng mã giả trên lưới 4×4, 9×9 và 16×16, kèm bảng số lần thử của 11 đề mẫu.

Về khả năng tiếp cận, các cặp màu chữ và nền đạt độ tương phản WCAG AA. Mỗi trạng thái ô có thêm dấu hiệu ngoài màu, ví dụ gạch ngang cho số bị loại và viền nét đứt cho ô vừa trả lại. Lưới đi được bằng phím mũi tên. Trang có chế độ tối, có bố cục riêng cho điện thoại, và hoạt ảnh ở trang chủ đứng yên khi máy bật giảm chuyển động.

Em chưa kiểm thử với người dùng thật. Các góp ý về giao diện và câu chữ đến từ việc em tự dùng thử, cộng với các phiên Claude và agent đóng vai người dùng phổ thông, sinh viên và biên tập viên (Chương 6). Đây là hạn chế lớn nhất của phần đánh giá giao diện.

8.6 Hạn chế và hướng phát triển

Hạn chế

- Để minh họa từng bước, bộ giải trên website chậm hơn bản đệ quy thường từ 63 đến 135 lần tùy đề, 135 lần ở đề Inkala. Theo tốc độ đo trên Chrome, đề chống quay lui cần khoảng 14 phút khi bấm Giải ngay.
- Chưa có người dùng thật thử.
- MRV mới có ở trang So sánh, chưa chạy từng bước được ở trang Giải Sudoku.
- Trang Kích thước lưới chỉ có 11 đề mẫu cố định, chưa cho dán hay gõ đề riêng. Các đề 16×16 sinh ngẫu nhiên nên một số có nhiều lời giải.
- Đề ngẫu nhiên trong bộ testcase nghiêng về phía dễ, vì script bỏ các đề cần hơn 1 triệu lần thử.
- Đáp án của bộ testcase và bộ giải của website cùng do Claude viết (Chương 7).
- Lịch sử chỉ giữ 100 bước gần nhất. Muốn xem lại bước cũ hơn thì phải chạy lại từ đầu.
- Website chỉ có tiếng Việt, dù toàn bộ chữ đã tách sẵn ra một file để dịch.

Hướng phát triển

- Cho trang Giải Sudoku chọn MRV và chạy từng bước.
- Chạy Giải ngay bằng bản đệ quy thường trong một luồng riêng của trình duyệt (Web Worker), chỉ dùng generator khi cần xem từng bước.
- Cho trang Kích thước lưới nhận đề do người dùng dán, và thêm các cỡ khác như 25×25.
- Dùng lại phần minh họa cho các bài quay lui khác trong slide: 8 quân hậu, tổng tập con, tô màu đồ thị.
- Thử với một nhóm người học thật, ghi lại chỗ họ không hiểu.
- Thêm bản tiếng Anh.

Phụ lục A

Chạy mã nguồn

A.1 Cấu trúc bài nộp

Bài nộp có bốn thư mục con theo yêu cầu của cô:

- `Source code/`: toàn bộ thư mục `final-project` của repo GitHub, không kèm lịch sử git, để các lệnh bên dưới chạy được nguyên vẹn.
- `Video demo/`: file `sudoku-backtracking.mp4`.
- `Testcase/`: bản sao của `testcases/`, gồm 6 file đề theo nhóm, file `manifest.csv`, file đề cho lưới kích thước khác và các script sinh đề.
- `Report/`: mã nguồn LaTeX của report và file PDF.

A.2 Địa chỉ website và mã nguồn

Website chạy trực tiếp trên trình duyệt, không cần cài gì. Mã nguồn có trên GitHub và trong thư mục Source code của bài nộp:

- Website: https://solving-sudoku-by-backtracking.vercel.app/
- GitHub: https://github.com/honglax/solving-sudoku-by-backtracking

Bên trong `Source code/` có bốn thư mục chính:

- `website/`: website React và TypeScript, kèm test đơn vị, test trên Chrome và script chụp ảnh cho report.
- `checks/`: các bản Python tham chiếu và file đáp án JSON.
- `testcases/`: 6 file đề theo nhóm, file `manifest.csv` ghi đáp án và số đo của từng đề, script sinh đề và script chạy bộ testcase.
- `report/`: mã nguồn LaTeX của report, dữ liệu đo, bảng sinh tự động và các script đo.

A.3 Các lệnh chạy

Cần Node 24, Yarn 4.18.1 (bật bằng Corepack), Python 3.9 trở lên và Chrome. Các lệnh chạy trong thư mục `website/`:

```
corepack enable
yarn install
yarn dev                          mở website ở http://localhost:5173
yarn test                         test đơn vị
yarn test:e2e                     test trên Chrome
yarn testcases                    292 đề nhanh của bộ testcase
yarn testcases --include-slow     cả 294 đề, khoảng 19 phút
yarn build                        bản build trong website/dist
```

Sinh lại bộ testcase và đáp án, chạy ở thư mục gốc:

```
python3 testcases/tools/generate_testcases.py
python3 checks/reference_solver.py
```

Biên dịch report trong thư mục `report/`:

```
python3 tools/build_report_tables.py
latexmk -xelatex main.tex
```

Tài liệu tham khảo

[1] Huỳnh Thị Thanh Thương, *CS112_0708. Quay lui (Backtracking), Nhánh cận (Branch and Bound)*, slide bài giảng môn Phân tích và thiết kế thuật toán, Trường Đại học Công nghệ Thông tin, 2026.

[2] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed., Pearson, 2012. Mục 12.1 Backtracking.

[3] P. Norvig, *Solving Every Sudoku Puzzle*, https://norvig.com/sudoku.html.

[4] Wikipedia, *Sudoku solving algorithms*, https://en.wikipedia.org/wiki/Sudoku_solving_algorithms.

[5] Wikipedia, *Sudoku*, https://en.wikipedia.org/wiki/Sudoku.

[6] Wikipedia, *Mathematics of Sudoku*, https://en.wikipedia.org/wiki/Mathematics_of_Sudoku.

[7] Wikipedia, *Graph coloring*, https://en.wikipedia.org/wiki/Graph_coloring.

[8] A. Stuart, *Arto Inkala Sudoku*, SudokuWiki, https://www.sudokuwiki.org/Print_Arto_Inkala_Sudoku.

[9] A. Stuart, *AI Escargot*, SudokuWiki, https://www.sudokuwiki.org/Escargot.

[10] Sudoku Snake, *Easter Monster*, https://www.sudokusnake.com/eastermonster.php.